Nayax Marshall Protocol

Proprietary & Confidential

IMPORTANT NOTICE
This document contains proprietary information of Nayax.
The information is Confidential and may be covered by non-disclosure agreements.

Table of content

Nayax Marshall Protocol....1
1. Scope....5
2. Definitions....6
3. System Overview....8
4. Device's Hardware and Software Limitations....8
5. Device's Internal SW Architecture....9
6. Marshall Protocol Overview....10
6.1 Baud Rate....10
6.2 Packet Structure....10
6.3 Basic SW Layers Architecture....12
7. Device – Peripheral Pairing Process....14
8. Marshall API – General Purpose Functions....15
8.1 Response....15
8.2 Reset....15
8.3 Firmware Info....16
8.4 Config....17
8.5 Keep Alive....18
8.6 Display Message....19
8.7 Display Control....19
8.8 Display Status....22
8.9 Display Event....22
8.10 Transfer Data....23
8.11 Status....26
QR Code....29
8.12 Get Time....30
8.13 Time....30
8.14 Get Random Number....30
8.15 Random Number....30
8.16 Set Peripheral Parameter....31

8.17 Set Payment Parameters 31
8.18 Modem Status 33
8.19 Open Socket 34
8.20 Close Socket 34
8.21 Send Data 35
8.22 Receive Data 35
8.23 Modem Rx Control 35
8.24 Trace 36
8.25 Alert 36
8.26 MDB Command 37
8.26.1 Session Type 38
8.26.2 Prepaid Pay 38
8.26.3 Close Session 39
8.26.4 Get Sessions Status 39
8.26.5 Sessions Status 39
8.26.6 Multi Vend Request 40
8.26.7 Multi Vend Success 40
8.26.8 Get Reader Status 42
8.26.9 Reader Status 42
8.26.10 Selection Request 43
8.26.11 Selection Denied 44
9. Marshall API – External Reader dedicated functions 46
9.1 RDR Set Event 47
9.2 RDR Event 47
9.3 RDR Transaction Request 48
9.4 RDR Dispense Confirm 49
9.5 RDR Transaction Cancel 49
9.6 RDR End Transaction 49
10. Marshall API – EV Charge Point Dedicated Functions 50
10.1 EV Charge Point Boot 50
10.2 EV Charge Point Status 51

10.3 EV Charge Point Start 52
10.4 EV Charge Point Stop 52
10.5 EV Charge Point Meter Value 53
10.6 EV Charge Point Diagnostics 54
11. Marshall API – EV Meter App Dedicated Functions 55
11.1 EV Meter App Get Info 55
11.2 EV Meter App Info 56
11.3 EV Meter App Charging 56
11.4 EV Meter App Status 57
11.5 EV Meter App Set Grid Params 59
11.6 EV Meter App Grid Params 60
12. Marshall API Protocol - Summary 61
13. Device – Peripheral Communication Examples 63
13.1 Pairing process example 63
13.2 Keep Alive Example 63
14. Vending Process 64
14.1 Device High Level State Machine 64
14.2 Supported Vend Sessions (MDB commands) 65
14.3 Reader enable/disable commands 69
15. External Reader (PIN Pad) Transaction Process 70
15.1 Important Note 70
15.2 Supported Vend Sessions (External Reader commands) 70

1. Scope

This document describes the API of the proprietary "***Marshall***" protocol supported by the Nayax' Payment and Communication Device.
This protocol is especially designed to be used by a 3rd party Peripheral (e.g. PC Machine / EV Charger / External Card Reader) to be integrated into the Nayax' Payment System.

2. Definitions

DCS	Device Configuration System Nayax' server back office for: - Payment devices' configuration, management and monitoring - Billing providers' configuration and assignment to machines' operator - Transactions + reports - Telemetry + reports - Alerts and Events management - Prepaid cards' management
Device	Nayax' payment and communication Device. e.g. Amit / VPOS Touch / Onyx / Fusion / … Device is able to communicate with DCS **only**!
Peripheral	- Vending Machine - EV Charger - External Reader e.g. VPOS Verify / PINPad / Tablet / …
Pre-selection Transaction	Consumer first selects a product. Settlement is done on product's price. If approved, machine can dispense the product. Also called "One step transaction".
Pre-authorization Transaction	Consumer presents first card, authorized on maximal amount. If approved, machine is activated; settlement is done on the final price. Also called "Two steps transaction".
Pairing process	To "introduce" a new Peripheral to the Device.
Keep Alive	A heartbeat mechanism for Peripheral to keep connection with Device. Should it not be sent on time, the Pairing process is automatically re-initiated by Device.
VMC	Vending Machine Controller
NAMA	National Automatic Merchandising Association
MDB	Multi Drop Bus – common standard protocol for Vending Machines. Marshall's transaction flow is based on the MDB's flow. Refer to *"NAMA – MDB Version 4.2* *Section 7 – Cashless Devices"*
FTL	File Transport Layer Part of the MDB standard. Refer to *"NAMA – MDB Version 4.2* *Section 7 – Cashless Devices"*
UID	Unique ID of a Prepaid card
SHA	Secured Hash Algorithm. Used to generate Signature of some raw data.

Single Vend	Classic transaction flow: - *Press START button* (Mandatory in Europe only) - Choose product - Pay Machine to dispense product/start service
Multi Vend	A shopping cart feature. Suits vending machines which can dispense multiple products in one transaction e.g. Smart Fridges.
Remote Vend	Allows the customer to select a product from a mobile application instead of the perihperal
Multi Sessions	A transaction working in two separate parts: 1) Pre-authorization 2) Settlement Suits EV Chargers, Smart Lockers etc. where final product price is either unknown or transaction may last longer than 4 minutes. - Transactions can stay "open" up to 23 hours. - Up to 60 transactions can be active simultaneously. - According to Nayax' policy, Authorized transactions with no Settlement are automatically settled after 24 hours.
Session Timeout 1380 minutes	By default, a multi-session timeout is set to 60 minutes. In case a transaction needs to be open for longer, Nayax needs to change this attribute accordingly.
Info Session	Allows transmission of a credit card/NFC card number (UID) to the Peripheral. e.g. Identification verification, car park systems, members discounts etc.
Reader Always On	The reader is never Disabled by the Peripheral and a transaction may start by presenting a card. Suits peripherals requiring to present card first in order to start the service, e.g. Smart Fridges where a successful card presentation opens the door.
Always Idle	Transaction is triggered by selecting first a product on the machine. No need in the START button or presenting a card first to the reader. In MDB translation: no need for the reader to send the "Begin Session" – the machine will directly send the "Vend Request" to launch a transaction.
EV	Electrical Vehicle
EV Charge Point	Electrical Vehicle Charging Station
EVSE	Electrical Vehicle Supply Equipment

3. System Overview

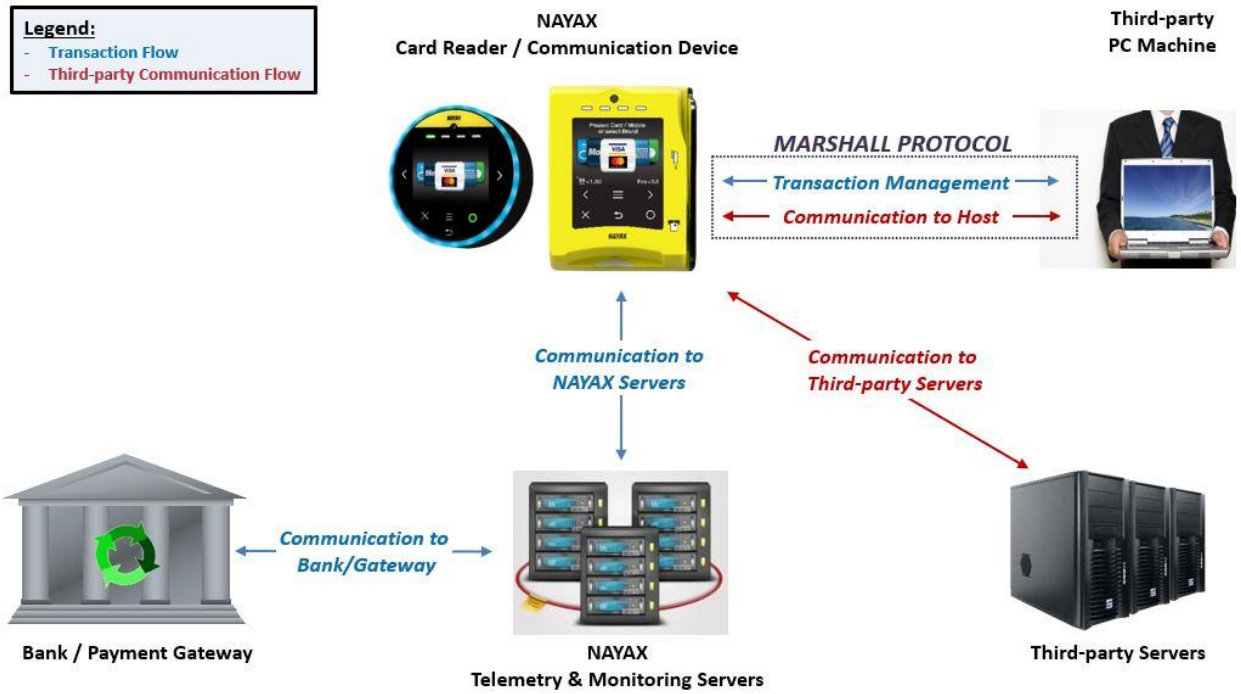


Figure 1 – Marshall Vending, System Overview

4. Device's Hardware and Software Limitations

The proprietary "***Marshall***" protocol has been designed to enable many Peripherals to be connected to one Device.
The Device can manage up to 16 different Peripherals in Software.
However, the Device is limited to up to 3 Serial Ports available in Hardware.

Should a Marshall integration need more than 3 Peripherals to be connected to a unique Device, Nayax will advise on a solution, if any.

5. Device's Internal SW Architecture

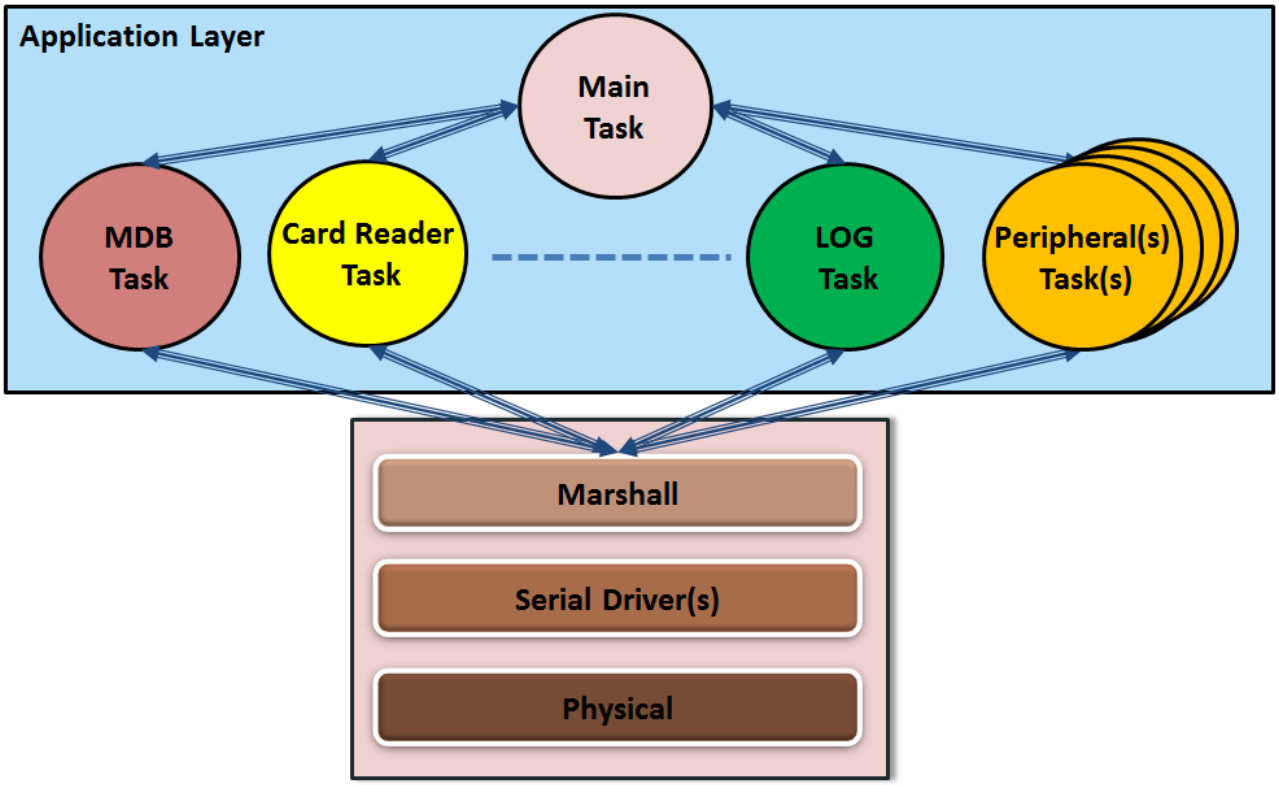


Figure 2 – Device SW Architecture

❖ Application Layer

The Application layer contains the following tasks:

- **Main** task, in charge of low priority operations
- **MDB** task
- **Card Reader** task
- **Peripherals** dedicated tasks
- **Log** task

❖ Marshall Layer

The Marshall layer is in charge of

- The Peripheral's database management
- Handling the Receive / Transmit packet from / to the Serial driver.

6. Marshall Protocol Overview

6.1 Baud Rate

Default baud rate of communication over a serial line between Device and Peripheral is:

- 115200 bps
- 8 bits
- No parity

6.2 Packet Structure

Packet Length	*Packet Options*	*Packet ID*	*Source*	*Source LSB*	*Dest.*	*Dest. LSB*	*Function Code*	*Data*	*CRC16*
2 Bytes	1 Byte	1 Byte	1 Byte	1 Byte	1 Byte	1 Byte	1 Byte	n Bytes	2 Bytes

Figure 3 – Packet Structure

Packet Length:	Length of Packet Message	(2 bytes)
Packet Options:	Packet Options bitmap	(1 byte)
Packet ID:	Packet ID – Running Identifier	(1 byte)
Source:	Source of Packet	(1 byte)
Source LSB:	LSB Serial Number of Source	(1 byte)
Destination:	Destination of Packet	(1 byte)
Destination LSB:	LSB Serial Number of Destination	(1 byte)
Function Code:	Specific Code of Function	(1 byte)
Data:	Data of Message	((*Packet Length* – 9) bytes)
CRC16:	Signature of Packet	(2 bytes)

6.2.1 Packet Length

Size of *Packet Length* field (= 2 bytes) is not included in Packet Length calculated value!

6.2.2 Packet Options

Bitmap:

Bit 0:	1 – ACK to Packet Requested
Bit 1:	1 – Packet encrypted
Bits 2-3:	Retry number (0 means 1st transmission)
Bit 4:	Is Last packet (0 means last)
Bits 5-7:	*RFU*

6.2.3 Packet ID

Running identifier of packet, wrapping out at 0xFF = 255.
Initial value can be any value in range 0-255 (starting with 0 is not mandatory).
In case a packet is missing, i.e. 2 consecutive packets sent by the same Peripheral have non-consecutive IDs, Device will reset the specific Peripheral and this will re-synchronize the communication between Device and Peripheral.

6.2.4 Source / Destination

The Device is **always** identified by ID 0x00.
Each Peripheral communicating with Device performs a pairing process (described below) with Device, in order to determine its specific ID.
At the 1st communication, the Peripheral identifies itself by ID 0xFF.
Once a Peripheral has been assigned an ID by the Device, it will identify itself with this ID until its next Power Cycle.

6.2.5 Broadcast Messages

Only Device is allowed to send Broadcast messages.
A Broadcast message is marked with 0xFF in the *<Destination>* field of the packet.

Broadcast messages are not required to be ACK'ed by the Peripherals
(i.e. Bit 0 in *<Packet Options>* field is 0).

6.2.6 Response to Message (ACK / NAK)

The receiver of the message should check that it is the correct recipient, as well as the CRC of the message, and sends a response with:

- ACK (= 0x00) if correct,

or

- NAK (≠ 0x00) if incorrect, and disregard the message

The transmitter will send a message and wait for either ACK or NAK.
If either NAK is received, or timeout, then the message is re-transmitted up to 3 times, and aborted after the third NAK is received / timeout.
The interval between retries is randomly valued at 50 – 100 milliseconds calculated by the sender.

6.2.7 Endianess

The Marshall protocol is little-endian i.e. any parameter bigger than 8 bits (Packet Length, CRC, …) is in little-endian format.

6.3 Basic SW Layers Architecture

6.3.1 Marshall Layer

The Marshall Layer is responsible for the Peripheral's database management, therefore:

- At Receive check:
  - ✓ Source and Destination are correct
  - ✓ Packet ID is the expected one
  - ✓ If ACK is required, send Response with ACK/NAK to sender
  - ✓ If OK, signal the task that a packet is waiting

- At Transmit:
  - ✓ Add Source and Destination
  - ✓ Add Packet ID
  - ✓ If ACK is required, set ACK bit in packet options

The Marshall Layer is also responsible for:

- ✓ Receive / Transmit packet from / to Serial driver
- ✓ Manage retry mechanism of transmission
- ✓ Check / Build CRC of packet (see below)
  Note: CRC is calculated on ALL the packet's fields.

```
//This function calculates CRC-16 CCITT
#define UTILS_POLYNOMIAL_CCITT           0x1021   /* x^16 + x^12 + x^5 + 1 */
#define UTILS_SEED_CCITT                 0

unsigned short Utils_CalcCrcCcitt ( unsigned char      *pData,
                                    unsigned int         Len,
                                    unsigned short       Seed )
{
    register unsigned int        i, j;
             unsigned short      Temp, ShortC, CurrentCRC = UTILS_SEED_CCITT, Crc = Seed;

    for ( i = 0; i < Len; i++ )
    {
        ShortC = (pData[i] & 0x00FF);
        Temp   = ((Crc >> 8) ^ ShortC) << 8;

        for ( j = 0; j < 8; j++ )
        {
            if ( ((CurrentCRC ^ Temp) & 0x8000) )
                CurrentCRC = ((CurrentCRC << 1) ^ UTILS_POLYNOMIAL_CCITT);
            else
                CurrentCRC <<= 1;
            Temp <<= 1;
        }

        Crc = ( (Crc << 8) ^ CurrentCRC );
    }

    return ( Crc );
}
```

Figure 4 – CRC Calculation (refer to ISO 3309)

7. Device – Peripheral Pairing Process

- At Power-Up, both Device and Peripheral need to initiate a pairing process.
- Peripheral is locked until the pairing process is successfully achieved.
- When receiving the *Config()* command, the Peripheral should check only the destination LSB for address verification**.**
- After the pairing process, both Peripheral and Device will reject a packet that doesn't have the correct source and destination (respective ID and LSB).
- **In case of error in the communication**, like receiving a non-consecutive packet ID or not receiving a response for all transmission retries, the Peripheral **should stop sending any packet** (including *KeepAlive()*) and wait for the Device to send a *Reset()* command.

Device		Peripheral
Reset()	→	
	←	Firmware Info()
Config()	→	

Figure 5 – Pairing Process

8. Marshall API – General Purpose Functions

8.1 Response

- *Description:* Contains ACK / NAK response to message.
- *Direction:* Device ←→ Peripheral
- *Function Code:* 0x00
- *Parameter:* Response value:
  - 0 – ACK
  - 1 – CRC error
  - 2 – Wrong recipient
  - 3 – Non-consecutive Packet ID
  - 10 – Unexpected error
- *Note:* <Packet ID> field of *Response()* packet is same as <Packet ID> of message.

8.2 Reset

- *Description:* To Software reset the Peripheral.
  This function is the trigger to the Pairing process.
- *Direction:* Device → Peripheral
- *Function Code:* 0x01
- *Parameter:* None
- *Response:* *FirmwareInfo()* function (see below).

8.3 Firmware Info

- *Description:* Contains specific Firmware details of Peripheral.
  Sent as response to the *Reset()* function.
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0x05
- *Parameter:*
  1. Protocol Version (Major. Minor) (*2 bytes*)
  2. Peripheral Type (*1 byte*)
     Refer to *E_MarshallApi_FirmInfoType* in "MarshallApi.h" file.
  3. Peripheral SubType (*1 byte*)
     NOTE: This field is no longer in use
  4. Peripheral Capabilities bitmap (*2 bytes - bitmap*)
     Refer to *Peripheral Capabilities bitmap* in "MarshallApi.h" file.
     See table below for further explanations.
  5. Peripheral Model (*20 bytes - ASCII*)
  6. Peripheral Serial Number (*20 bytes - ASCII*)
  7. Peripheral App. SW Version (*20 bytes - ASCII*)
  8. Peripheral HW Version (*20 bytes - ASCII*)
  9. Peripheral Manufacturer Code (*20 bytes - ASCII*)
  10. Marshall SDK Version (*20 bytes - ASCII*)
- *Response:* *Config()* function (see below).
- *Note:* Peripheral Capabilities bitmap:

Bit	Description
0	**Magstripe.** Dedicated to **Marshall Reader** to support reading cards with Magnetic stripe.
1	**Contact.** Dedicated to **Marshall Reader** to support reading cards with Smart chip in Contact.
2	**Contactless.** Dedicated to **Marshall Reader** to support reading cards with Smart chip in Contactless.
3	**Keypad.** Dedicated to **Marshall Reader** with a Keypad.
4	**Display.** Dedicated to **Marshall Reader** with a Display.
5	**Touch screen.** Dedicated to **Marshall Reader** with a Touch screen.
6	**Multi session.** Peripheral works in Multi sessions mode.
7	**Peripheral will approve Contactless proprietary cards.** Peripheral is connected to its server and authorizes its proprietary Contactless cards. NOTE: Once this bit is set, system will support 3rd party proprietary cards ONLY! i.e. there is no possibility to accept other proprietary cards.

8	**Vend request price is not final.** Combined with Multi session support bit, enables a Pre-Authorization transaction flow.
9	**Peripheral will approve Magnetic stripe proprietary cards.** Peripheral is connected to its server and authorizes its proprietary Magstripe cards. NOTE: Once this bit is set, system will support 3rd party proprietary cards ONLY! i.e. there is no possibility to accept other proprietary cards.

8.4 Config

- *Description:* Set parameters for communication.
  Sent as response to the *FirmwareInfo()* function.
- *Direction:* Device ➔ Peripheral
- *Function Code:* 0x06
- *Parameter:*
  1. Protocol Version (Major. Minor) (*2 Bytes*)
  2. Peripheral ID (source/destination address) (*1 byte*)
  3. Keep Alive Interval in milli Seconds (*4 bytes*)
     Default Value: 1000 (= 1 second)
  4. Language (*1 byte*)
     Default Value: 0 – English
  5. Country Code (According to ISO standard) (*3 bytes*)
  6. Currency Code (According to ISO standard) (*3 bytes*)
  7. Decimal Place (*1 byte*)
  8. Device's Serial No (*16 bytes*)
  9. Maximum Message Length (*2 bytes*)
     Default Value: 512 Bytes
     NOTE: Includes Header and Footer of Marshall's message.
- *Response:* None

8.5 Keep Alive

- *Description:* Indication that the Peripheral is connected.
  Sent every X mSec (default is 1 second) if no activity.
  i.e. not during a transaction nor during communication.
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0x07
- *Parameter:* None
- *Response:* None
- *Notes:*
  1. Any packet sent by the Peripheral is considered as "*Keep Alive*" by the Device.
  2. If two *"Keep Alive"* are not consecutively sent, the Device will interpret it as the Peripheral is disconnected and will:
     a. Send an Alert to Nayax' server that Peripheral is disconnected.
     b. Re-initiate a Pairing process by sending the *Reset()* command to the Peripheral.

8.6 Display Message

- *Description:* Display Message on Peripheral's LCD.
- *Direction:* Device ➔ Peripheral
- *Function Code:* 0x08
- *Parameter:* 1. Message to be displayed (*ASCII string*)
  2. Picture file ID (*2 bytes*)
     If 0xFFFF, no picture to display
  3. Audio file ID (*2 bytes*)
     If 0xFFFF, no sound to play

  *The following fields are available from version **2.00** and above:*

  4. QR Code (*ASCII string*)
  5. Barcode (*ASCII string*)
- *Response:* *DisplayStatus()* function (see below).
- *Note:* Recipient is responsible for correct display of message.
  If message is longer than display's capabilities, the last x bytes of the message will be discarded.
  The Device may supersede the Peripheral's message with its own message.

8.7 Display Control

- *Description:* Control display of Nayax' Device.
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0x14
- *Parameter:* Specific data in TLV mode. (*N bytes*)
  TLV stands for Tag Length Value where:
  - Tag (*1 byte*)
  - Length (*1 byte*)
  - Value (*Length bytes*)

  Refer to *E_MarshallApi_DisplayControlTags* in "MarshallApi.h" file.
- *Response:* *DisplayStatus()* function (see below).
- *Notes:* 1. Tags shall be defined by Nayax (see below).
  2. Customization will be defined by Nayax per customer.

<table>
<tr><th>Tag</th><th>Tag Name</th><th>Data Size</th><th>Data Type</th><th>Description</th></tr>
<tr><td>1</td><td>Layout ID</td><td>2 bytes</td><td>Integer</td><td>Layout ID as defined by Nayax.
<u>NOTE 1:</u> If Layout involves logic, the logic is controlled by the Nayax' Device.
Example: Vertical scrolling screen.
Peripheral will supply the whole information to enable Device to control the N screens to be displayed during scrolling operation.
<u>NOTE 2:</u> If message is longer than display's capabilities, the last X bytes of the message will be discarded.</td></tr>
<tr><td>2</td><td>Label</td><td>N bytes</td><td>ASCII</td><td>Message/Title/Product/Amount to be displayed.
Label is displayed following its numbering in the specific Layout.
Structure of Label is:
- Object ID in Layout 1 byte
- Label Number in Layout 1 byte
- Label String (N - 2) bytes
<u>NOTE:</u> Device's limitation is up to 12 Objects per Layout.</td></tr>
<tr><td>3</td><td>Button(s) ID(s)</td><td>N bytes</td><td>Integer</td><td><u>Optional:</u> If no such tag, no buttons are enabled.

Device's identifier of button(s) to be pressed by consumer following Layout's logic.
Same Button ID could be returned to Peripheral once pressed by consumer for flow control.

Possible values:
<table>
<tr><th>Button</th><th>ID</th></tr>
<tr><td>< (Left Arrow)</td><td>100</td></tr>
<tr><td>≡ (Menu)</td><td>101</td></tr>
<tr><td>> (Right Arrow)</td><td>102</td></tr>
<tr><td>X (Cancel)</td><td>103</td></tr>
<tr><td>↩ (Back)</td><td>104</td></tr>
<tr><td>O (Enter / OK)</td><td>105</td></tr>
<tr><td>Touch Screen</td><td>106</td></tr>
</table></td></tr>
</table>

Tag	Tag Name	Data Size	Data Type	Description
4	Primary Language	1 byte	Integer	Primary language for text and audio messages. Value is as below: English = 0, Hebrew = 1, Romanian = 2, Norwegian = 3, Icelandic = 4, Portuguese = 5, Spanish = 6, Italian = 7, Swedish = 8, Danish = 9, Polish = 10, Brazilian = 11, French = 12, Dutch = 13, Catalan = 14, Swiss German = 15, Hungarian = 16, Chinese = 17, German = 18, Finnish = 19, Euskera = 20, Estonian = 21, Czech = 22, Turkish = 23, Latvian = 24, Lithuanian = 25, Japanese = 26, Arabic = 27, Slovakian = 28, Slovenian = 29, Welsh = 31, Croatian = 32, Ukrainian = 33, Russian = 34,
5	Secondary Language	1 byte	Integer	Secondary language for text messages to be displayed alternatively with Primary's. Value is same as Primary language (see above).

8.8 Display Status

- *Description:* Response to the *DisplayMessage()* and *ControlDisplay()* functions.
- *Direction:* Device ←→ Peripheral
- *Function Code:* 0x09
- *Parameter:* Display status (*1 byte*):
  0 – Display OK
  1 – Display error
- *Response:* None

8.9 Display Event

- *Description:* Event in return to the *DisplayControl()* function.
- *Direction:* Device → Peripheral
- *Function Code:* 0x15
- *Parameter:* Specific data in TLV mode. (*N bytes*)
  TLV stands for Tag Length Value where:
  - Tag (*1 byte*)
  - Length (*1 byte*)
  - Value (*Length bytes*)

  Refer to *E_MarshallApi_DisplayEventTags* in "MarshallApi.h" file.
- *Response:* None
- *Note:* Tags shall be defined by Nayax (see below).

Tag	Tag Name	Data Size	Data Type	Description
1	Button ID	1 Byte	Integer	Device's identifier of button pressed by consumer following Layout's logic. Possible values: **0..99** (Object ID) — 0..99 < (Left Arrow) — 100 ≡ (Menu) — 101 > (Right Arrow) — 102 X (Cancel) — 103 ↩ (Back) — 104 O (Enter / OK) — 105 Touch Screen — 106
2	Keyboard Data	N Bytes	ASCII	String of up to **3** digits entered over the virtual Keyboard. Refers to the "Keyboard Layout" option. **NOTE: This feature is conditioned by a flag defined in DCS**

8.10 Transfer Data

- *Description:* Transfer specific data.
- *Direction:* Device ←→ Peripheral
- *Function Code:* 0x0A
- *Parameter:* Specific data in TLV mode. (*N bytes*)
  TLV stands for ***T***ag ***L***ength ***V***alue where:
  - Tag (*1 byte*)
  - Length (*1 byte*)
  - Value (*Length bytes*)

  Refer to *E_MarshallApi_TransferDataTags* in "MarshallApi.h" file.
- *Response:* None
- *Note:* Tags shall be defined by Nayax (see below).

Tag	Tag Name	Data Size	Description
1	Transaction ID	8 bytes	Device/Peripheral Transaction ID.
2	“Choose Product” Timeout	2 bytes	Peripheral can update Device’s timeout “Choose Product Timeout”. If set to zero, the Device will not play nor display the “Choose Product” message. This parameter can only be updated when Device is in Idle state and NOT during a session.
3	Card Type	1 byte	Session’s Card Type (refer to E_MarshallApi_TDCardType in “MarshallApi.h” file).
4	Card Entry Mode	1 byte	Session’s Card Entry Mode (refer to E_MarshallApi_TDCardEntryType in “MarshallApi.h” file).
5	Card BIN	6 bytes	Card’s BIN (first 6 digits – ASCII).
6	Card PAN Hash	20 bytes	SHA1 (= Signature) of the Card’s PAN.
7	Proprietary Card UID	MiFare – 4,7,11 bytes Mag – up to 40 bytes	Proprietary Card’s UID (= *U*nique *Id*entifier).
8	VMC Authorization Status	1 byte	Card’s authorization status (Peripheral to Device). 0 – Approved 1 – Declined
9	COM status	1 byte	Server – Device communication status.
10	FTL Data	N (up to 500) bytes	Raw Data passed over FTL (= *F*ile *T*ransfer *L*ayer) like in MDB Level 3.
11	Create Layout	4 bytes	Layout
12	Update Layout	N (up to 500) bytes	Update layout by ExtraData field (4 bytes). Tag – last/not, extra data (0 -not last 1 -last 2 – extra data).
13	Credit card four last digits	4 bytes	Credit card’s four last digits (in ASCII)
14	Product Code	2 bytes	Product code as defined in VMC/DCS.
15	Product Price	4 bytes	Product price as defined in VMC/DCS.
16	Customized data / General purpose	N (up to 500) bytes	Customized data in format JSON. Dictionary: TBD.
17	Consumer ID	N (up to 500) bytes	ID #/Passport #/… ASCII string.
18	Main software version	14 bytes	The device's Main software version as defined in the DCS
19	POS software version	10 bytes	The device's POS software version as defined in the DCS
20	Default credit	2 bytes	The "Default credit" as defined in the DCS
21	Monyx ID	9 bytes	The "Monyx ID" as defined in the DCS

Transfer Data format by card types:

Credit Card	Proprietary Card
Tag 3: Card Type (value in range "2"-"6") Tag 4: Entry Mode Tag 5: Card BIN Tag 6: Card PAN Hash Tag 13: Credit card last four digits Tag 1: Transaction ID Tag 9: COM Status	Tag 3: Card Type (value of "1") Tag 4: Entry Mode (value of"1" if Magstripe, "4" if MiFare, "5" if HID) Tag 7: Proprietary Card UID Tag 1: Transaction ID Tag 9: COM Status

8.11 Status

- *Description:* Status message.
- *Direction:* Device ←→ Peripheral
- *Function Code:* 0x0B
- *Parameter:* 1. Status ID (*1 byte*)
  See table below.
  2. Status Additional Data Length (*2 bytes*)
  3. Status Additional Data (*<Data Length> bytes*)
  In case of pending data, it will be equal to the available server data size.
  In case of settlement status, it will be zero for success and different than zero for error.
- *Response:* None
- *Note:* Fields *"Status Additional Data Length"* and *"Status Additional Data"* are optional and exist only whenever the *"Status ID"* needs them.

Status ID	Description
0	Unexpected error.
1	Timeout: any process timeout.
2	Out of sequence: Received out of sequence command.
3	Pending 3rd party communication data received.
4	Settlement status (Response to a “Close Session” command in multi session mode).
5	Machine status.
6	FTP status.
7	Machine Inhibit
8	Close session timeout
The Statuses below are available from version 3.20 and above	
20	Device in Idle mode, available for starting a transaction
	Bitmap of extra information (set in “Status Additional Data” field): Bit 0 – Ethernet connection is available
21	Device not available
	Bitmap of reasons (set in “Status Additional Data” field): Bit 0 – Device with cellular issues Bit 1 – Device with no network communication available Bit 2 – Device busy (long communication e.g. remote SW update) Bit 3 – Device in technician mode Bit 4 – Device with internal issue
The Statuses below are available from version 3.00 and above	
54	Call your Bank
56	Card Error (Reader not able to read card)
61	Insert Card into the Contact slot
62	Card not Accepted
65	Processing error (Card has been removed before completion of the transaction in Contact)
66	Remove Card from the Contact slot
67	Use Contact Reader
68	Use Magnetic Stripe Reader
69	Try again
71	Present Card
73	Card read OK
74	Insert card in Contact or Swipe
75	Present ONE card only
78	Use another Card
79	Insert card in Contact
82	Look at your mobile (consumer is completing the transaction on its mobile phone)
83	Present Card Again

84	Insert or Swipe another card

QR Code

- *Description:* Transmit a QR code data to 3rd party peripheral.
- *Direction:* Device ➔ Peripheral
- *Function Code:* 0x11
- *Parameter:* 1. QR code type (*1 byte*)
  - 0 – Dynamic QR
  - 1 – Static eReceipt
  - 2 – Dynamic eReceipt

  Refer to *E_MarshallApi_QrDataTypeTag* in "MarshallApi.h" file.

  2. QR code data (*up to 500 bytes*)
- *Response:* None

8.12 Get Time

- *Description:* Retrieve Current Time data.
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0x0C
- *Parameter:* None
- *Response:* *Time()* function (see below).

8.13 Time

- *Description:* Time.
- *Direction:* Device ➔ Peripheral
- *Function Code:* 0x0D
- *Parameter:*
  1. Year, 0-99 (*1 byte*)
  2. Month, 1-12 (*1 byte*)
  3. Day, 1-31 (*1 byte*)
  4. Hours, 0-23 (*1 byte*)
  5. Minutes, 0-59 (*1 byte*)
  6. Seconds, 0-59 (*1 byte*)
- *Response:* None

8.14 Get Random Number

- *Description:* Request to generate a Random Number.
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0x12
- *Parameter:* None
- *Response:* *RandomNumber()* function (see below).
- *Note:* Could be used as Unique ID / Session ID / Transaction ID.

8.15 Random Number

- *Description:* Random Number.
- *Direction:* Device ➔ Peripheral
- *Function Code:* 0x13
- *Parameter:* Random Number (*32 bytes*)
- *Response:* None
- *Note:* Intrinsically based on Time Stamp + Random Number.

8.16 Set Peripheral Parameter

- *Description:* Set Peripheral parameter set in DCS Nayax' back office.
- *Direction:* Device ➔ Peripheral
- *Function Code:* 0x0E
- *Parameter:* 1. Parameter ID (*2 bytes*)
  2. Parameter Data (*n bytes*)
- *Response:* None

8.17 Set Payment Parameters

- *Description:* Set specific Payment Parameters required in order for the Peripheral to process the Transactions vs. their own Acquirer or PSP.
  Parameters are defined in DCS Nayax' back office.
- *Direction:* Device ➔ Peripheral
- *Function Code:* 0x0F
- *Parameter:* Specific data in TLV mode. (*N bytes*)
  TLV stands for ***T***ag ***L***ength ***V***alue where:
  - Tag (*1 byte*)
  - Length (*1 byte*)
  - Value (*Length bytes*)
- *Response:* None
- *Notes:* 1. Tags shall be defined by Nayax (see below).
  2. In case of Multi Applications environment, this command will be sent X times according to the number of Payment Applications running within the Peripheral.
  Tag *"Application ID"* acts as the identifier of the dedicated parameters.

Tag	Tag Name	Data Size	Description
0	Application ID	String	Payment Application ID. Used in multi-application environment. This ID is also the Priority of the Application to be selected for clearing the Transaction.
1	Acquirer ID	String	Can be more than one, separated by “;”
2	Merchant ID	String	Merchant ID.
3	Terminal ID	String	Terminal ID allocated by the Acquirer.
4	Machine Location	String	Can be more than one, separated by “;”
5	MCC	String	Merchant Category Code.
6	Acquirer Server Port	String	Can be more than one, separated by “;”
7	Acquirer Server Address	String	Can be more than one, separated by “;”
8	Technician password	String	Technician password
9	Merchant Name	String	Merchant Name
10	EU VAT number	String	EU VAT number
11	Merchant Acceptor Identifier	String	Merchant Acceptor Identifier
12	Merchant Acceptor Address	String	Merchant Acceptor Address
13	Transaction currency	String	Transaction currency
14	Transaction type	String	Transaction type
15	Transaction Result	String	Authorized/Declined Transaction Result
16	Transaction Authorization Code	String	Transaction Authorization Code
17	Dedicated File Name	String	Dedicated File Name
18	Application Preferred Name or Application Label	String	Application Preferred Name or Application Label
19	Card Holder Identification Method	String	Card Holder Identification Method

8.18 Modem Status

- *Description:* This is a response to all modem commands.
- *Direction:* Device ➔ Peripheral
- *Function Code:* 0x20
- *Parameter:*
  1. Command (*1 byte*)
     The command the modem is responding to
     (0 in case it is NOT a response).
  2. Modem's return value (*1 byte*)
     - 0 – OK
     - 1 – Fail
     - 2 – Busy
     - 3 – Parameter error
     - 4 – Command error
     - 5 – Socket is closed
     - 128 – No available resource
     - 129 – Unsupported socket type
  3. Status extra data (*1 byte*)
     In case returned value is 2 (busy) it will contain the busy time in seconds; Peripheral **should wait accordingly** before sending another modem command.

8.19 Open Socket

- *Description:* Open a socket for communication with a server.
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0x21
- *Parameter:* 1. Destination (*1 byte*)
  0 – Nayax' Server
  1 – 3rd party's Server
  2. Protocol (*1 byte*)
  0 – TCP
  1 – UDP **(Not supported yet)**
  2 – FTP
  3. Server Port (*2 bytes*)
  4. Server details
  In case of TCP socket this field will contain the
  IP address in format: 111.222.333.444 (*ASCII characters*)
  or DNS address in format: abcde.com (*ASCII characters*)
  Use **Null-character** to signify end of string (IP or DNS).
  In case of FTP socket this field will contain the server,
  user name, password and file names separated by a semicolon.
  Use **Null-character** to signify end of string.
  For example: "111.222.333.444;User;Password;FileName.txt"
  5. Extra Data (*4 bytes*) – Optional (supported from Rev **2.0**)
  - Buffer Length (*2 bytes*)
    Value shall be between 100 up to 400 Bytes.
    Default value is 400 Bytes.
  - Open Socket Timeout in seconds (*2 bytes*)
    Value shall be between 1 up to 120 seconds.
    Default value is 60 seconds.

  Important Note: If using extra data, need to add all 4 bytes even if only one of the attributes is needed.
  Fill the 2 bytes of the unneeded attribute with 0.
- *Note:* If destination is the Nayax' Server, Device will use its default Port and default IP address.
- *Response:* *ModemStatus()* function (see above).

8.20 Close Socket

- *Description:* Close the socket.
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0x22
- *Parameter:* None
- *Response:* *ModemStatus()* function (see above).

8.21 Send Data

- *Description:* Send Data message to the opened socket.
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0x23
- *Parameter:* Data (*<Data Length> bytes*).
- *Note:* Due to modem's limitation, the largest packet to be sent at a time shall not exceed **400 bytes** length.
  Should you need to send largest data, you need to split them in chunks of 400 bytes each.
- *Response:* *ModemStatus()* function (see above).

8.22 Receive Data

- *Description:* Device will send the data received from the opened socket.
- *Direction:* Device ➔ Peripheral
- *Function Code:* 0x24
- *Parameter:* Data (*<Data Length> bytes*).
- *Note:* Due to modem's limitation, the largest packet to be received at a time shall not exceed **400 bytes** length.
  Should you need to receive largest data, you need to split them in chunks of 400 bytes each.
- *Response:* None

8.23 Modem Rx Control

- *Description:* Used for flow control: Peripheral can pause/resume the Modem's activity while receiving data.
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0x25
- *Parameter:* Flow Control (*1 byte*)
  0 – Resume
  1 – Pause
- *Response:* *ModemStatus()* function (see above).

8.24 Trace

- *Description:* Send Trace parameters to Device.
  The Peripheral's Trace will be inserted within the existing Nayax' "GTrace".
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0x30
- *Parameter:* 1. Trace number (*2 bytes*)
  2. Number of Parameters (*1 byte*)
  Possible values: 0, 1 or 2
  3. Parameters (Numeric only) (*2 bytes per parameter*)
- *Response:* None
- *Note:* Specific string of Trace and its number will be defined in "C" language printf format by operator in DCS.
  Example(with one parameter):
  Trace number – 11001 – "Reader Trace no: %d\n"

8.25 Alert

- *Description:* Send Alert to Device.
  Alert can be forwarded to Nayax' server.
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0x31
- *Parameter:* 1. Alert ID (*2 bytes*)
  2. Data of Alert (*N bytes – ASCII string*)
- *Response:* None
- *Note:* New Alerts should be submitted by 3rd party to Nayax, to be defined on Nayax' Server.

8.26 MDB Command

- *Description:* Pass MDB standard command.
- *Direction:* Peripheral ←→ Device
- *Function Code:* 0x80
- *Parameter:* MDB Command as defined in:
  "NAMA – MDB Version 4.2
  Section 7 – Cashless Devices"
  Device supports "MDB Levels 1 and 2" and
  "MDB Level 3": FTL and 'Always Idle' features only.
- *Response:* According to MDB protocol.
- *Note:* Extended proprietary commands:

General Extended MDB commands:

8.26.1 Session Type

Direction: Peripheral ➔ Device

Command	Code	Sub-Command	Response
Vend	13H	81H – Session Type	None

Vend (13H)	Session Type command (81H) - X1	Session Type X2

Session Type: 0 – Vending.
1 – Info.

Note: When a prepaid card is used within an Info session its card number and card type will be stored so we can use it in future Pay command call.
Up to one card number is stored at any given time (The last one that was stored).

8.26.2 Prepaid Pay

Direction: Peripheral ➔ Device

Command	Code	Sub-Command	Response
Vend	13H	83H – Prepaid Pay	None

Vend (13H)	Prepaid Pay command (83H) - X1	Card number string up to 128 Bytes

Card number – Sent as extra verification. This value should match the card number that the Device stored in the last info session and will be used in the current Prepaid Pay command.
Should the card number be different, the Prepaid Pay command will fail.

Note: Once Prepaid Pay command has been issued, the card number that was stored in the Device is cleared.

Extended MDB commands used in Multi Sessions mode:

8.26.3 Close Session

Direction: Peripheral ➔ Device

Command	Code	Sub-Command	Response
Vend	13H	80H – Close Session	None

Vend (13H)	Close Session command (80H) - X1	Session Status X2	Product Price X3-X4	Product Code X5-X6	Quantity X7-X8	Unit of measure X9

Session Status: 0 – Status OK.
1 – User cancel.
2 – Failed to dispense the product.
3 – Timeout.
4 – Vend denied.

Product Code = Session ID.

8.26.4 Get Sessions Status

Direction: Peripheral ➔ Device
Note: This function is available from version **3.20** and above

Command	Code	Sub-Command	Response
Vend	13H	85H – Get Sessions Status	Sessions Status

Vend (13H)	Get Sessions Status (85H) - X1

8.26.5 Sessions Status

Direction: Device ➔ Peripheral
Note: This function is available from version **3.20** and above

Command	Sub-Command	Response
Vend	86H – Sessions Status	None

Sessions Status (86H) - X1	Number of Opened Sessions X2	Opened Session 1 X3-X4	…	Opened Session N

Extended MDB commands used in Multi-Vend mode:

8.26.6 Multi Vend Request

Direction: Peripheral ➔ Device
Note: Up to 20 products in list are supported.

Command	Code	Sub-Command	Response
Vend	13H	82H – Multi Vend Request	None

Vend (13H)	Multi Vend Request (82H) - X1	Product Code X2-X3	Product Price X4-X5	Quantity X6-X9	Unit of measure X10

Quantity – To specify multiple selections of the same item.
Unit of measure – Grams/Liter/Items etc.

Example:

Multi Vend Request for 2 products:

Vend (13H)	Multi Vend Request (**82H**) - X1	Code **1** X2-X3	Price **1** X4-X5	Quantity **1** X6-X9	Unit **1** X10	Code **2** X11-X12	Price **2** X13-X14	Quantity **2** X15-X18	Unit **2** X19

When each costing 1.00 (100 with decimal place of 2) it would look like this:
13:82:00:00:64:00:01:00:00:00:01:01:00:64:00:01:00:00:00:01:

8.26.7 Multi Vend Success

Direction: Peripheral ➔ Device
Note: Up to 20 products in list are supported.

Command	Code	Sub-Command	Response
Vend	13H	84H – Multi Vend Success	None

Vend (13H)	Multi Vend Success (84H) - X1	Product Code X2-X3	Product Price X4-X5	Quantity X6-X9	Unit of measure X10

Quantity – To specify multiple selections of the same item.
Unit of measure – Grams/Liter/Items etc.
Note: "Extended Multi Vend Success" must either have the same product list as "Extended Multi Vend Request" or a smaller list (in case of items that failed to dispense).

Example:

Multi Vend Success for 2 products:

Vend (13H)	Multi Vend Success (**84H**) - X1	Code **1** X2-X3	Price **1** X4-X5	Quantity **1** X6-X9	Unit **1** X10	Code **2** X11-X12	Price **2** X13-X14	Quantity **2** X15-X18	Unit **2** X19

When each costing 1.00 (100 with decimal place of 2) it would look like this:
13:84:00:00:64:00:01:00:00:00:01:01:00:64:00:01:00:00:00:01:

Extended MDB commands used for Reader:

8.26.8 Get Reader Status

Direction: Peripheral ➔ Device
Note: This function is available from version **3.20** and above

Command	Code	Sub-Command	Response
Reader	14H	87H – Get Reader Status	Reader Status

Reader (14H)	Get Reader Status (87H) - X1

8.26.9 Reader Status

Direction: Device ➔ Peripheral
Note: This function is available from version **3.20** and above

Command	Sub-Command	Response
Reader	88H – Reader Status	None

Reader Status (88H) - X1	Status X2

Reader Status: 0 – Reader Disabled.
1 – Reader Enabled.

Extended MDB commands used in Remote Vend mode

8.26.10 Selection Request

Direction: Device ➔ Peripheral
Note: This function is available from version **3.21** and above

Command	Sub-Command	Response
Remote Vend	14H - Selection Request	None

Selection Request (14H)	Funds Available X1-X2	Payment Media ID X3-X6	Payment Type X7	Payment Data X8-X9	Item Number X10-X11	Item Options X12-X15

Funds Available:
0x0000 Irrelevant
0x0001-0xFFFE Credit amount
0xFFFF Undefined credit

Payment Media ID: 0xFFFFFFFF

Payment Type:
00xxxxxxb = Normal vend card
x1xxxxxxb = Test media
1xxxxxxxb = Free vend card
xx000000b - 0 VMC default prices
xx000001b - 1 User Group
Price list number
xx000010b - 2 User Group
Discount group index
xx000011b - 3 Discount percentage factor
xx000100b - 4 Surcharge percentage factor

Payment data: 0x0000

Item number: The item number to be dispensed.
Item number is defined by the Peripheral and is equivalent to the product code of VEND REQUEST

Item options: bit 0: 0
(The rest of the bits are irrelevant)

8.26.11 Selection Denied

Direction: Peripheral ➔ Device

Note: This function is available from version 3.21 and above

Command	Code	Sub-Command	Response
Vend	13H	07H – Selection Denied	None

Vend (13H)	07H – Selection Denied X1	Item Number X2-X3	Reason X4

Item Number: Should be equal to "Item Number" of the "Selection Request" command.

Reason:
0x00: Unknown error / out of order.
0x01: Selection doesn't exist / not configured.
0x02: Selection empty / product was not dispensed.
0x03: Selection defective.
0x04: Ingredient is over.
0x05: Selection is blocked.
0x06: Selection is inhibited (for this time or this pricelist or this user).
0x07: Product is expired.
0x08: Temperature conditions is out of range.
0x09: Machine is busy or is in the wrong mode (menu for example).
0x0A: Insufficient credit.
0xFF: Machine is temporarily busy.

Comparison between standard MDB and MDB over Marshall

Standard MDB	MDB over Marshall
9600 bps, 11 bits (1 start bit, 8 data bits, 1 mode bit, 1 stop bit)	115200 bps, 8 data bits, No parity
Single session	Multi session
Limited multi vend capability (up to 5 products)	Multi vend
Max message payload over FTL: 36 Bytes	Max message payload: 500 bytes
Master/Slave Protocol, POLL - ACK	Bidirectional protocol, event driven
POLL every 25 - 200 milliseconds	Keep Alive in configurable interval
ACK within 5 milliseconds	ACK conditioned by Timeout
Response upon POLL only	Response conditioned by Timeout
	Extended proprietary commands

9. Marshall API – External Reader dedicated functions

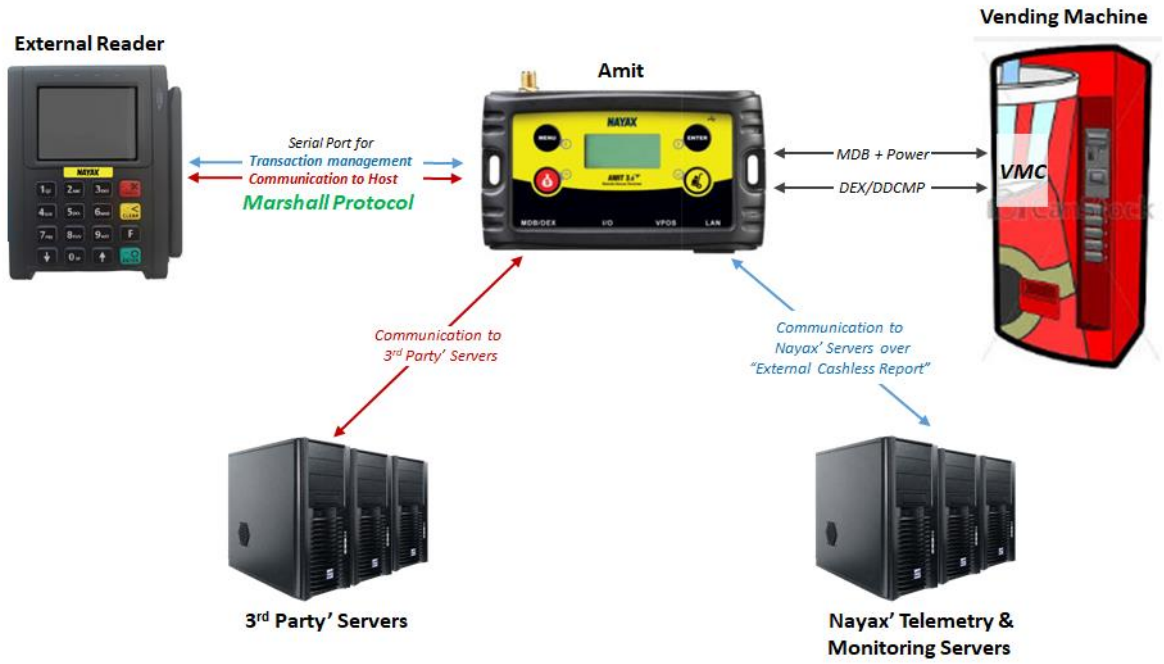


Figure 6 – Marshall External Reader, System Overview

The following functions are used by the Nayax' Amit Device in order to drive a transaction performed over an External Reader.

9.1 RDR Set Event

- *Description:* Set event to occur on External Reader.
- *Direction:* Amit ➔ External Reader
- *Function Code:* 0x90
- *Parameter:* Event Type Activation – bitmap (*1 byte*)
  - Bit 0 – Keyboard
  - Bit 1 – Swipe
  - Bit 2 – Contact
  - Bit 3 – Contactless
  - Bit 4 – Contactless NFC
  - Bit 5 – Mobile Application
- *Response:* *RdrEvent()* function (see below).
- *Note:* Amit will endlessly wait for the *RdrEvent()* response.

9.2 RDR Event

- *Description:* Event type occurred on External Reader and its specific data.
- *Direction:* External Reader ➔ Amit
- *Function Code:* 0x91
- *Parameter:* 1. Event Type (*1 byte*)
  - 0 – Key Pressed
  - 1 – Swipe
  - 2 – Contact
  - 3 – Contactless
  - 4 – Contactless NFC
  - 5 – Mobile Application

  2. Payment Type OR Key Value (*1 byte*)

Payment Type	Key Value
1 – Credit Card Magstripe	0
2 – Credit Card Contact EMV	1
3 – Credit Card Contactless EMV	2
4 – Credit Card Contactless Magstripe	3
5 – Contactless MiFare	4
6 – Contactless HID	5
7 – QR Code	6
11 – Prepaid Magstripe	7
12 – Prepaid Contact	8
13 – Prepaid Contactless	9
	0x0A – CLEAR
	0x0B – CANCEL
	0x0D – ENTER

0x10 – F
0x11 – F1
0x12 – F2
0x13 – F3
0x14 – F4

3. Payment Type Data (*1 byte*)
   1 – ASCII
   2 – Binary
   3 – BCD
   4 – Decimal
   5 – Hexadecimal
4. Payment Type Data Value (*N bytes*)
   - *Credit Card:*
     - BIN (*6 bytes*)
     - PAN 4 last digits (*4 bytes*)
     - Acquirer ID (*12 bytes*)
     - Transaction ID (unique, controlled by Bank) (*8 bytes*)
     - Time Stamp YYYYMMDDHHMMSS (*14 bytes*)

   *The following field is available from version **2.00** and above:*
     - SHA256 signature of PAN (*32 bytes*)
   - *Prepaid:*
     - User ID (*20 bytes*)
     - Optional Proprietary data (*up to 128 bytes*)

   *The following option is available from version **2.00** and above:*
   - *QR Code:*
     - QR Data (*up to 400 bytes*)

- *Response:* None
- *Note:* Fields *"Payment Type Data"* and *"Payment Type Data Value"* exist **only** when *"Event Type"* value is NOT *Key Pressed*.

9.3 RDR Transaction Request

- *Description:* Provides the External Reader with the Transaction's parameters.
- *Direction:* Amit ➔ External Reader
- *Function Code:* 0x92
- *Parameter:* 1. Product Code (*2 bytes*)
  2. Product Price (*4 bytes*)
- *Response:* None

9.4 RDR Dispense Confirm

- *Description:* Indicates whether the product was dispensed correctly.
- *Direction:* Amit ➔ External Reader
- *Function Code:* 0x93
- *Parameter:* Result of Dispense (*1 byte*)
  - 0 – OK
  - 1 – Time Out
  - 2 – Canceled by user
  - 3 – Canceled by Vending Machine
  - 4 – No credit
  - 5 – Vending Machine Not Idle
  - 6 – Product can NOT be dispensed
  - 7 – Vending Machine powered down
  - 8 – Power Down during dispense
  - 9 – Vending Machine canceled during auth. process
- *Response:* None

9.5 RDR Transaction Cancel

- *Description:* Bi-directional command in order to cancel a transaction.
- *Direction:* External Reader ⬅➔ Amit
- *Function Code:* 0x94
- *Parameter:* Cancellation cause (*1 byte*)
  - 0 – Refund/Void Transaction
  - 1 – Transaction declined by bank
  - 2 – Bank's server communication Time Out
  - 3 – Wrong PIN code (after X retries)
  - 10 – Cancelled by consumer
  - 11 – Consumer Time Out
  - 12 – Vending Machine communication error
- *Response:* None

9.6 RDR End Transaction

- *Description:* Indicates end of transaction.
- *Direction:* External Reader ➔ Amit
- *Function Code:* 0x95
- *Parameter:* None
- *Response:* None

10. Marshall API – EV Charge Point Dedicated Functions

The following functions are used by an EV Charge Point e.g. Nayax' *EV Meter* to pass telemetry information to the Device.
Device will push this information to the Nayax' Server for payment and/or telemetry purposes.

Figure 7 – Marshall EV Charge Point, System Overview

10.1 EV Charge Point Boot

- *Description:* To notify Nayax' server of a Boot event.
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0xA0
- *Parameter:*
  1. Charge Point Vendor (*10 bytes*)
  2. Charge Point Model (*10 bytes*)
  3. Charge Point Serial Number (*20 bytes*)
  4. Charge Point SW Version (*20 bytes*)
  5. Charge Point HW Version (*20 bytes*)
  6. Charge Point Charging Profile (*1 byte*)
  7. Number of EVSE (*1 byte*)
  8. Boot Reason (*1 byte*)
     - 1 – Application Reset
     - 2 – Firmware Update
     - 3 – Local Reset
     - 4 – Power Up
     - 5 – Remote Reset
     - 6 – Scheduled Reset
     - 7 – Triggered
     - 8 – Unknown Reason
     - 9 – Watchdog
  9. Reserved for future use (*17 bytes*)

10. The following info is repeated as "Number of EVSE":
    a. EVSE ID (*1 byte*)
    b. EVSE Vendor (*10 bytes – ASCII*)
    c. EVSE Model (*10 bytes – ASCII*)
    d. EVSE Serial Number (*20 bytes – ASCII*)
    e. EVSE SW Version (*20 bytes – ASCII*)
    f. EVSE HW Version (*20 bytes – ASCII*)
    g. Plug Type (*20 bytes – ASCII*)
    h. Max Current Allowed (*2 bytes*)
    i. Reserved for future use (*17 bytes*)

- *Response:* None

10.2 EV Charge Point Status

- *Description:* To notify Nayax' server on current Charger's Status.
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0xA1
- *Parameter:*
  1. EVSE ID (*1 byte*)
  2. EVSE Connector ID (*1 byte*)
  3. Status (*1 byte*)
     1 – Available
     2 – Preparing
     3 – Charging
     4 – Suspended EV
     5 – Suspended EVSE
     6 – Finishing
     7 – Reserved
     8 – Unavailable
     9 – Faulted
  4. Vendor Error code (*2 bytes – bitmap*)
     Bit 0 – Relay Stuck
     Bit 1 – Relay Not Close
     Bit 2 – Error on RCD
     Bit 3 – Error on PE/N Wires
     Bit 4 – Over Voltage
     Bit 5 – Over Current
     Bit 6 – High Temperature
     Bit 7 – Unsupported Mode
     Bits 8-15 – Reserved for future use
- *Response:* None

10.3 EV Charge Point Start

- *Description:* To notify Nayax' server of a Start Charging event.
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0xA2
- *Parameter:*
  1. Session ID (*32 bytes*)
  2. EVSE ID (*1 byte*)
  3. EVSE Connector ID (*1 byte*)
  4. Current Session Energy in Wh (*4 bytes*)
  5. Global Energy in Wh (*4 bytes*)
- *Response:* None

10.4 EV Charge Point Stop

- *Description:* To notify Nayax' server of a Stop Charging event.
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0xA3
- *Parameter:*
  1. Session ID (*32 bytes*)
  2. EVSE ID (*1 byte*)
  3. EVSE Connector ID (*1 byte*)
  4. Current Session Energy in Wh (*4 bytes*)
  5. Global Energy in Wh (*4 bytes*)
  6. Stop Reason (*1 byte*)
     - 1 – Local
     - 2 – Remote
     - 3 – Emergency
     - 4 – No longer authorized
     - 5 – Cable disconnected
     - 6 – Power lost
     - 7 – Reboot
     - 8 – Soft reset
     - 9 – Hard reset
     - 10 – Unlock command sent by Central
     - 11 – Other
- *Response:* None

10.5 EV Charge Point Meter Value

- *Description:* Continuous monitoring of the ongoing Charging session. Notifications sent to Nayax' server at an interval of X seconds, configurable in DCS.
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0xA4
- *Parameter:*
  1. Session ID (*32 bytes*)
  2. EVSE ID (*1 byte*)
  3. EVSE Connector ID (*1 byte*)
  4. Current Session Energy in Wh (*4 bytes*)
  5. Global Energy in Wh (*4 bytes*)
  6. Measured Voltage (*6 bytes - 2 bytes per phase*)
     Depending on resolution defined in DCS
  7. Measured Current (*6 bytes - 2 bytes per phase*)
     Depending on resolution defined in DCS
  8. Measured Power (*6 bytes - 2 bytes per phase*)
     Depending on resolution defined in DCS
  9. Temperature in Celsius (*1 byte - Signed*)
  10. Charging State (*1 byte*)
      0 – Not Charging
      1 – Charging 1 Phase
      2 – Charging 3 Phases
  11. State (*1 byte*)
      0 – Not Connected
      1 – Connected
      2 – Wants to Charge
      3 – Needs to Ventilate
      4 – Error State
  12. Load Current Allowed in A (*4 bytes*)
- *Response:* None

10.6 EV Charge Point Diagnostics

- *Description:* Send diagnostics to Nayax' server.
- *Direction:* Peripheral ➔ Device
- *Function Code:* 0xA5
- *Parameter:* 1. Number of EVSE (*1 byte*)
  2. The following info is repeated as "Number of EVSE":
     a. EVSE ID (*1 byte*)
     b. EVSE Connector ID (*1 byte*)
     c. Charging State (*1 byte*)
        0 – Not Charging
        1 – Charging 1 Phase
        2 – Charging 3 Phases
     d. State (*1 byte*)
        0 – Not Connected
        1 – Connected
        2 – Wants to Charge
        3 – Needs to Ventilate
        4 – Error State
     e. Warning (*1 byte - bitmap*)
        Bit 0 – Relay B Not Close
        Bit 1 – low Voltage or Missing Phase
        Bit 2 – Grid Problem
        Bit 3 – Charging Slow – High Temperature
        Bit 4 – Error in Incoming Packet
     f. Error (*2 bytes - bitmap*)
        Bit 0 – Relay Stuck
        Bit 1 – Relay Not Closed
        Bit 2 – Error on RCD
        Bit 3 – Error on PE/N Wires
        Bit 4 – Over Voltage
        Bit 5 – Over Current
        Bit 6 – High Temperature
        Bit 7 – Unsupported Mode
        Bits 8-15 – Reserved for future use
     g. Reserved for future use (*13 bytes*)
- *Response:* None

11. Marshall API – EV Meter App Dedicated Functions

The following functions are used by the Amit in order to drive an EV Charge Point e.g. Nayax' Bee Meter.
Amit communicates with the Nayax' Bluetooth component driven by the Nayax' "*EV Meter*" Mobile Application.

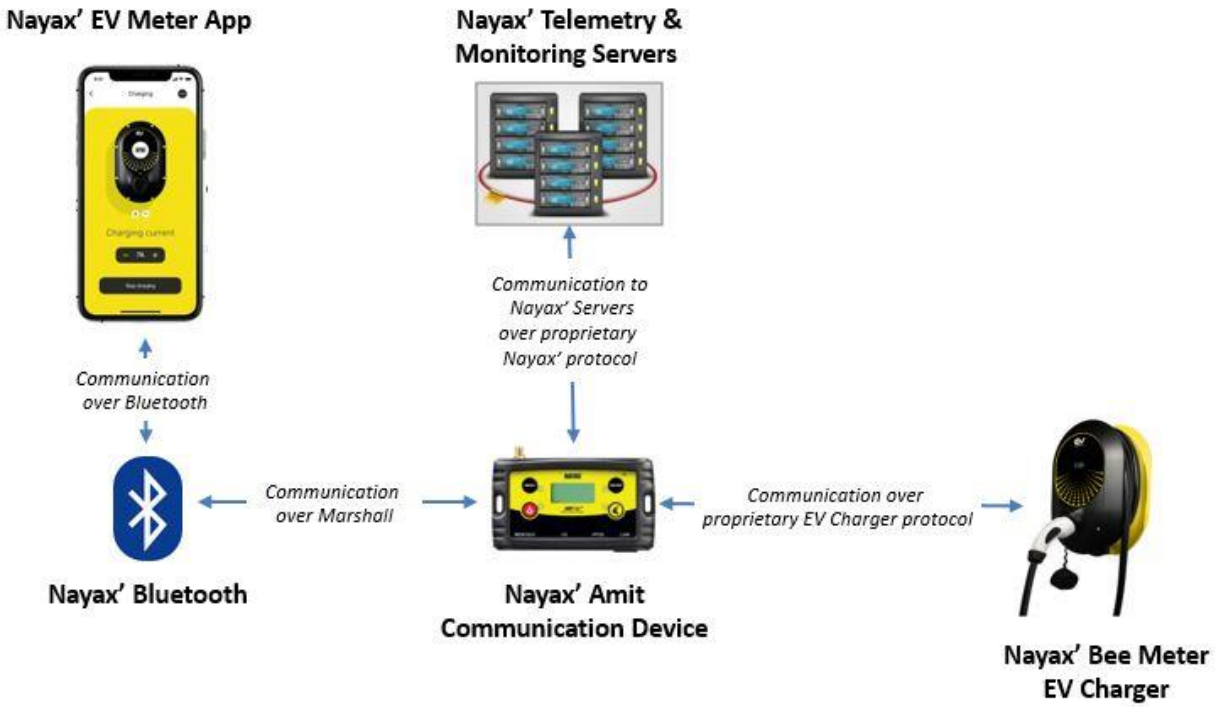


Figure 8 – Marshall EV Meter App, System Overview

11.1 EV Meter App Get Info

- *Description:* To retrieve EV Charger info.
- *Direction:* Bluetooth ➔ Amit
- *Function Code:* 0xE0
- *Parameter:* 1. User ID in Hexadecimal (*32 bytes*)
  2. Connector ID (*1 byte*)
- *Response:* *EVMeterAppInfo()* function, see below
- *Note:* This command shall be sent per number of EV Chargers.

11.2 EV Meter App Info

- *Description:* EV Charger Info.
- *Direction:* Amit ➔ Bluetooth
- *Function Code:* 0xE1
- *Parameter:* 1. Connector ID (*1 byte*)
  2. EV Charger Serial Number (*20 bytes*)
  3. EV Charger SW Version (*25 bytes*)
  4. EV Charger HW Version (*25 bytes*)
- *Response:* None

11.3 EV Meter App Charging

- *Description:* EV Start/Stop Charging command.
- *Direction:* Bluetooth ➔ Amit
- *Function Code:* 0xE2
- *Parameter:* 1. User ID in Hexadecimal (*32 bytes*)
  2. Connector ID (*1 byte*)
  3. Charge Command (*1 byte*)
     0 – Keep current state
     1 – Stop charging
     2 – Start Charging, 1 Phase
     3 – Start Charging, 3 Phases
  4. Current Allowed in Ampere (*1 byte*)
     0A or 6-32A
  5. Relay (*1 byte*)
     0 – Keep current state
     1 – Open
     2 – Close
- *Response:* *EVMeterAppStatus()* function, see below

11.4 EV Meter App Status

- *Description:* EV Charger's Status returned to *EVMeterAppCharging()* call.
- *Direction:* Amit ➔ Bluetooth
- *Function Code:* 0xE3
- *Parameter:*
  1. Connector ID (*1 byte*)
  2. State of Vehicle (*1 byte*)
     - 0 – Vehicle not connected
     - 1 – Vehicle connected
     - 2 – Vehicle wants to charge
     - 3 – Vehicle needs to ventilate
     - 4 – Error state
  3. State of charging (*1 byte*)
     - 0 – Not charging
     - 1 – Charging 1-phase
     - 2 – Charging 3-phases
  4. Warnings (*1 byte - bitmap*)
     - 0x00 – OK
     - 0x01 – Relay of phases 2 and 3 could not close
     - 0x02 – Low voltage or missing phase
     - 0x04 – Problem with grid
     - 0x08 – Charging slows down - high temperature (60-80° C)
     - 0x10 – Error in incoming packet
  5. Errors (*1 byte - bitmap*)
     - 0x00 – OK
     - 0x01 – Relay stuck closed
     - 0x02 – Relay could not close
     - 0x04 – Error on RCD
     - 0x08 – Error on PE/N wires
     - 0x10 – Overvoltage
     - 0x20 – Overcurrent
     - 0x40 – Temperature too high (> 80° C)
     - 0x80 – Unsupported charging mode
  6. Measured Voltage (*6 bytes - 2 bytes per phase*)
     Depending on resolution defined in DCS
  7. Measured Current (*6 bytes - 2 bytes per phase*)
     Depending on resolution defined in DCS
  8. Wh in session (*4 bytes*)
  9. All time Wh charged (*4 bytes*)

10. Locker State (*1 byte*)
    - 0 – Unknown
    - 1 – Trying to Lock
    - 2 – Trying to Unlock
    - 3 – Locked
    - 4 – Unlocked
    - 5 – Error
11. Temperature in Celsius (*1 byte - Signed*)
12. External Temperature in Celsius (*1 byte - Signed*)
13. Emergency Stop Button (*1 byte*)
    - 0 – Not Pressed
    - 1 – Pressed
    - 2 – Still Active

      i.e. button is no longer pressed, but not charging until car is unplugged and plugged in again.
14. Measured PP Resistance (*1 byte*)
    - 0 – < 220 ohm, 64A
    - 1 – 220 ohm, 32A
    - 2 – 470 ohm, 25A
    - 3 – 680 ohm, 20A
    - 4 – 1000 ohm, 16A
    - 5 – 15000 ohm, 13A
    - 6 – > 15000 ohm, 6A (plug not connected)
15. Power Factor (*3 bytes - 1 byte per phase*)

    Depending on resolution defined in DCS
16. Frequency in Hz (*6 bytes - 2 bytes per phase*)
17. Maximum Current Allowed in Ampere (*1 byte*)

    0A or 6-32A

- *Response:* None

11.5 EV Meter App Set Grid Params

- *Description:* Set EV Charger's Grid parameters.
- *Direction:* Bluetooth ➔ Amit
- *Function Code:* 0xE4
- *Parameter:* 1. Connector ID (*1 byte*)
  2. Grid Type (*1 byte*)
     0x00 – No change
     0x01 – TN-S
     0x02 – IT
     0x03 – USA-IT
  3. Grid Measurement (*1 byte*)
     0 – No change
     1 – From internal sensor (if equipped)
     2 – Wh from energy meter: 10 imp/kWh
     3 – Wh from energy meter: 100 imp/kWh
     4 – Wh from energy meter: 200 imp/kWh
     5 – Wh from energy meter: 400 imp/kWh
     6 – Wh from energy meter: 500 imp/kWh
     7 – Wh from energy meter: 800 imp/kWh
     8 – Wh from energy meter: 1000 imp/kWh
     9 – Wh from energy meter: 1600 imp/kWh
     10 – Wh from energy meter: 2000 imp/kWh
- *Response:* *EVMeterAppGridParams()* function, see below
- *Note:* Changing the Grid Type takes up to 2 seconds if all is OK.

11.6 EV Meter App Grid Params

- *Description:* EV Charger's current Grid parameters.
- *Direction:* Amit ➔ Bluetooth
- *Function Code:* 0xE5
- *Parameter:* 1. Connector ID (*1 byte*)
  2. Grid Type (*1 byte*)
     0x00 – Error
     0x01 – TN-S
     0x02 – IT
     0x03 – USA-IT
  3. Grid Measurement (*1 byte*)
     0 – Error
     1 – Internal data
     2 – Wh from energy meter: 10 imp/kWh
     3 – Wh from energy meter: 100 imp/kWh
     4 – Wh from energy meter: 200 imp/kWh
     5 – Wh from energy meter: 400 imp/kWh
     6 – Wh from energy meter: 500 imp/kWh
     7 – Wh from energy meter: 800 imp/kWh
     8 – Wh from energy meter: 1000 imp/kWh
     9 – Wh from energy meter: 1600 imp/kWh
     10 – Wh from energy meter: 2000 imp/kWh
- *Response:* None

12. Marshall API Protocol - Summary

Function Name	Function Code	Direction
Response	0x00	Device ↔ Peripheral
Reset	0x01	Device → Peripheral
Firmware Info	0x05	Peripheral → Device
Config	0x06	Device → Peripheral
Keep Alive	0x07	Peripheral → Device
Display Message	0x08	Device → Peripheral
Display Control	0x14	Peripheral → Device
Display Event	0x15	Device → Peripheral
Display Status	0x09	Peripheral → Device
Transfer Data	0x0A	Device ↔ Peripheral
Status	0x0B	Device ↔ Peripheral
QR Code	0x11	Device → Peripheral
Get Time	0x0C	Peripheral → Device
Time	0x0D	Device → Peripheral
Get Random Number	0x12	Peripheral → Device
Random Number	0x13	Device → Peripheral
Set Parameter	0x0E	Device → Peripheral
Set Payment Parameters	0x0F	Device → Peripheral
Modem Status	0x20	Device → Peripheral
Open Socket	0x21	Peripheral → Device
Close Socket	0x22	Peripheral → Device
Send Data	0x23	Peripheral → Device
Receive Data	0x24	Device → Peripheral
Modem Rx Control	0x25	Peripheral → Device

Function Name	Function Code	Direction
Trace	0x30	Peripheral → Device
Alert	0x31	Peripheral → Device
MDB Command	0x80	Device ↔ Peripheral
RDR Set Event	0x90	Device → External Reader
RDR Event	0x91	External Reader → Device
RDR Transaction Request	0x92	Amit → External Reader
RDR Dispense Confirm	0x93	Amit → External Reader
RDR Transaction Cancel	0x94	Amit ↔ External Reader
RDR End Transaction	0x95	External Reader → Amit
EV Charge Point Boot	0xA0	Peripheral → Amit
EV Charge Point Status	0xA1	Peripheral → Amit
EV Charge Point Start	0xA2	Peripheral → Amit
EV Charge Point Stop	0xA3	Peripheral → Amit
EV Charge Point Meter Value	0xA4	Peripheral → Amit
EV Charge Point Diagnostics	0xA5	Peripheral → Amit
EV Meter App Get Info	0xE0	Peripheral → Amit
EV Meter App Info	0xE1	Amit → Peripheral
EV Meter App Charging	0xE2	Peripheral → Amit
EV Meter App Status	0xE3	Amit → Peripheral
EV Meter App Set Grid Params	0xE4	Amit → Peripheral
EV Meter App Grid Params	0xE5	Peripheral → Amit

13. Device – Peripheral Communication Examples

13.1 Pairing process example

Device sends *Reset()*:
09 00 00 00 FF FF FF FF 01 6D 0F

Peripheral sends *FirmwareInfo()*:

Peripheral Type
Peripheral Model
Peripheral Serial Number
Protocol Version
Peripheral Capabilities

4B 00 00 00 02 7C 00 00 05 00 02 04 02 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72
73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F 80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92
93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F A0 A1 A2 A3 8D 1D

Peripheral Software Version

Device replies with *Config()*:

Keep Alive Poll Interval
Peripheral ID (packet source)
Currency Code
Language
Protocol Version
Amit Serial Number

2A 00 00 00 00 36 01 7C 06 00 02 01 E8 03 00 00 00 CC 03 00 00 00 00 00 02 36 30 37
31 36 31 30 30 30 30 30 30 30 30 30 30 00 02 67 2A

Country Code
Maximum Packet Length

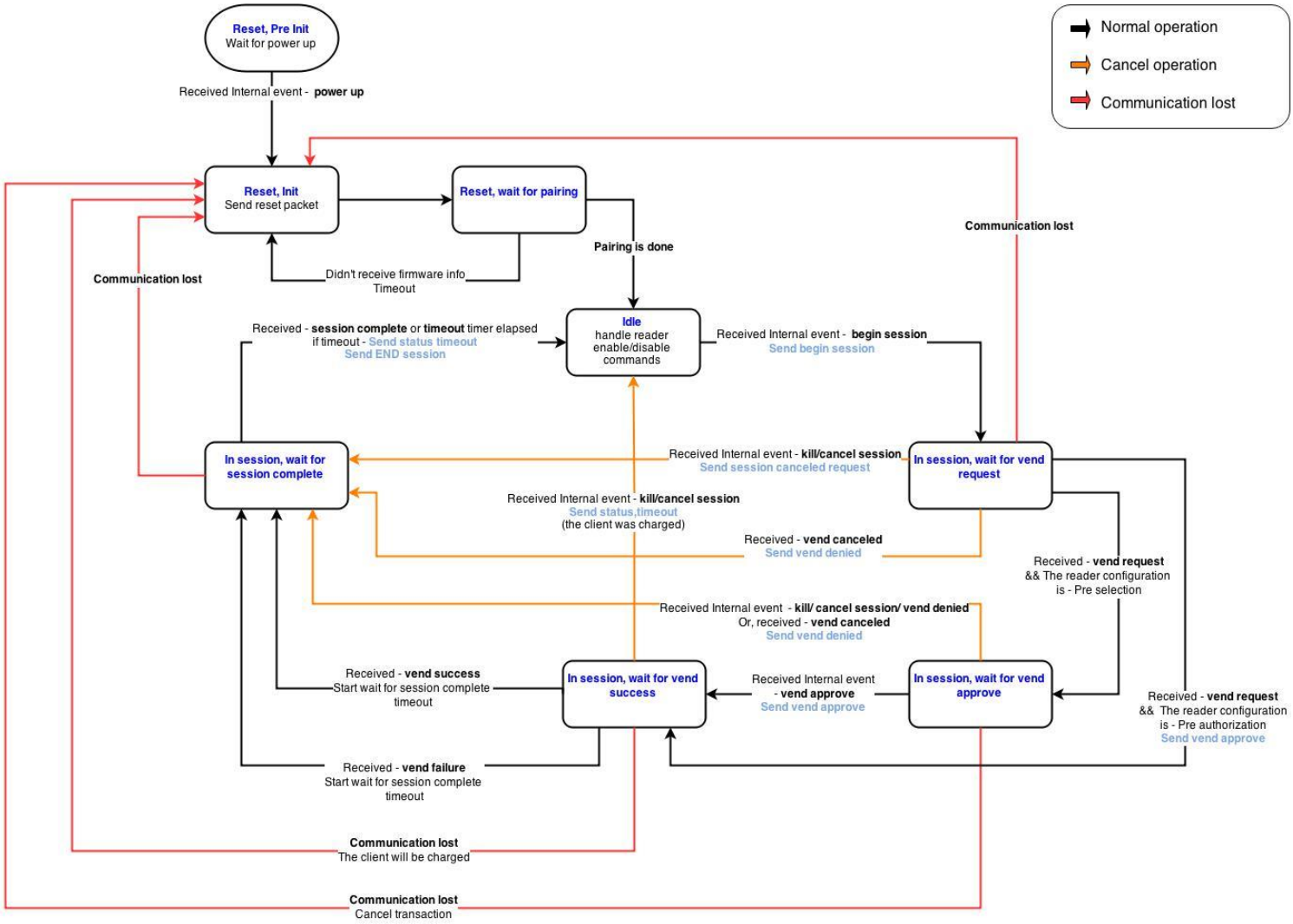
13.2 Keep Alive Example

Peripheral sends *KeepAlive()*:
09 00 01 30 01 53 00 30 07 F5 64

Device sends *Response(*ACK*)*:
0A 00 00 30 00 30 01 53 00 00 88 B0

14. Vending Process

14.1 Device High Level State Machine



14.2 Supported Vend Sessions (MDB commands)

14.2.1 Valid vend

Device		Peripheral
BEGIN SESSION	→	
		← VEND REQUEST
VEND APPROVE	→	
		← VEND SUCCESS
		← SESSION COMPLETE
END SESSION	→	

14.2.2 Session cancelled by user pressing Device's "Cancel" button

Device		Peripheral
BEGIN SESSION	→	
Consumer presses "Cancel" button		
SESSION CANCEL	→	
		← SESSION COMPLETE
END SESSION	→	

14.2.3 Session cancelled by Peripheral

Device	Peripheral
BEGIN SESSION →	
	Consumer cancels the session
	← SESSION COMPLETE
END SESSION →	

14.2.4 Session cancelled by Peripheral after "Vend Request"

Device	Peripheral
BEGIN SESSION →	
	← VEND REQUEST
	Consumer cancels the session
	← CANCEL VEND
VEND DENIED →	
	← SESSION COMPLETE
END SESSION →	

14.2.5 Peripheral failed to dispense product

Device		Peripheral
BEGIN SESSION	→	
		← VEND REQUEST
VEND APPROVE	→	
		← VEND FAILURE
		← SESSION COMPLETE
END SESSION	→	

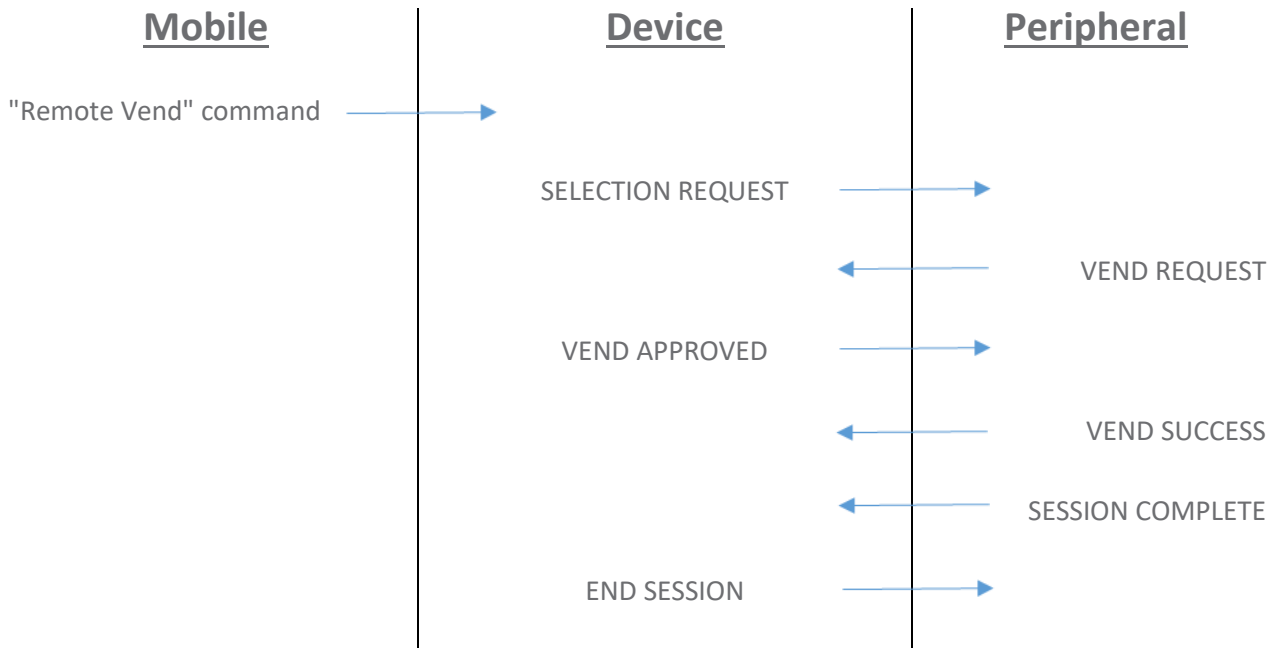
14.2.6 Vend denied by Device

Device		Peripheral
BEGIN SESSION	→	
		← VEND REQUEST
Insufficient funds or payment media/account error		
VEND DENIED	→	
		← SESSION COMPLETE
END SESSION	→	

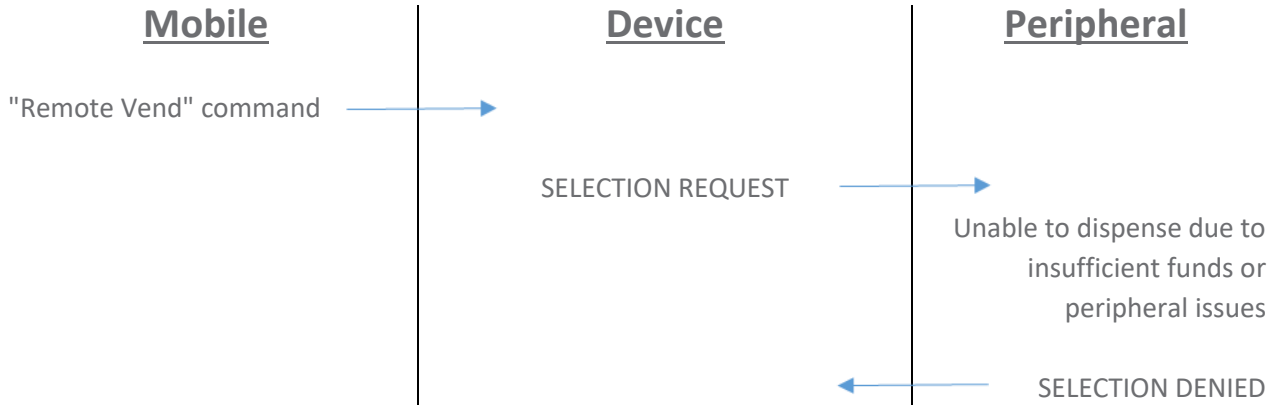
14.2.7 Always idle (Start with product selection) - Valid vend

Device		Peripheral
		← VEND REQUEST
VEND APPROVE	→	
		← VEND SUCCESS
		← SESSION COMPLETE
END SESSION	→	

14.2.8 Remote Vend – Valid vend



14.2.9 Remote Vend – Vend denied



14.3 Reader enable/disable commands

The Peripheral can enable or disable the reader using the MDB commands.
However in order to keep a valid session process, the Peripheral may change the reader state only when the Device is in an idle state, i.e. before the beginning of the session or after the session was completed, and the communication status is OK (the Pairing process was achieved successfully).

When the Device is set to "**Always On"** state the Peripheral should enable the reader after every successful pairing process.

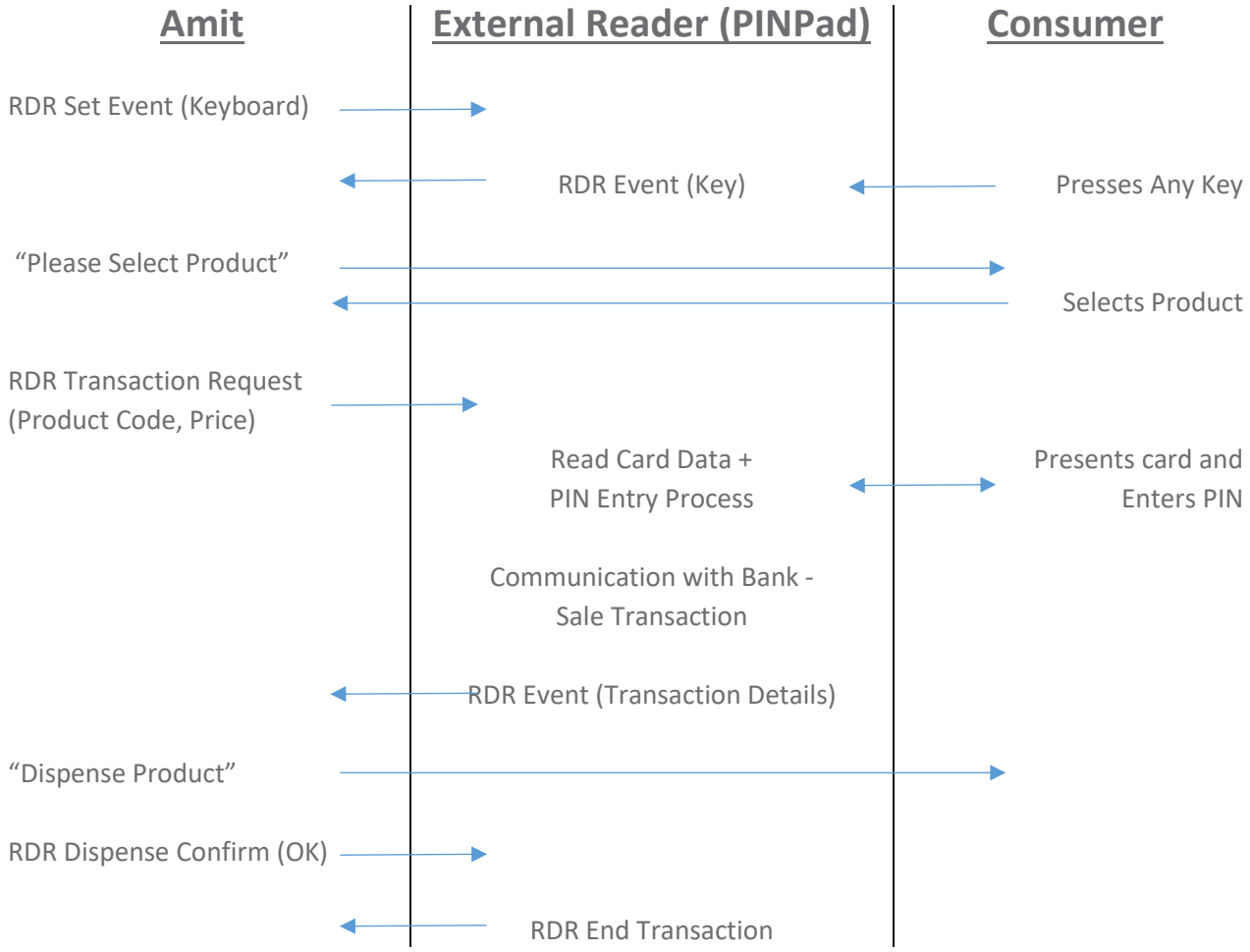
15. External Reader (PIN Pad) Transaction Process

15.1 Important Note

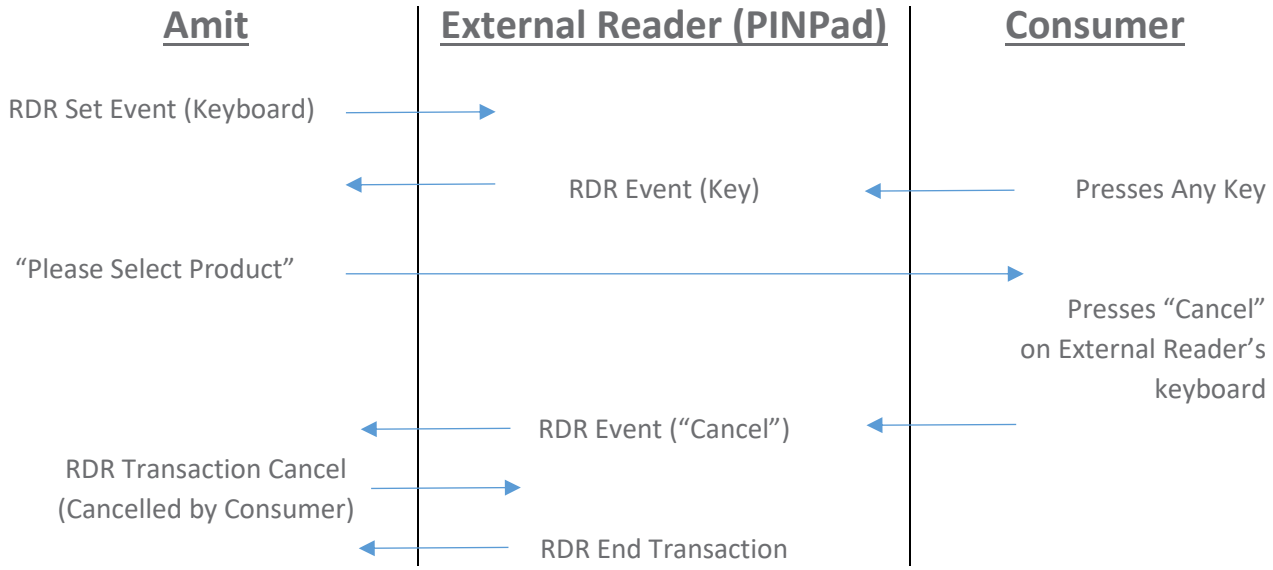
During the Transaction, Device should fully control the External Reader's display **except** during the process of Card Data read and PIN entry.

15.2 Supported Vend Sessions (External Reader commands)

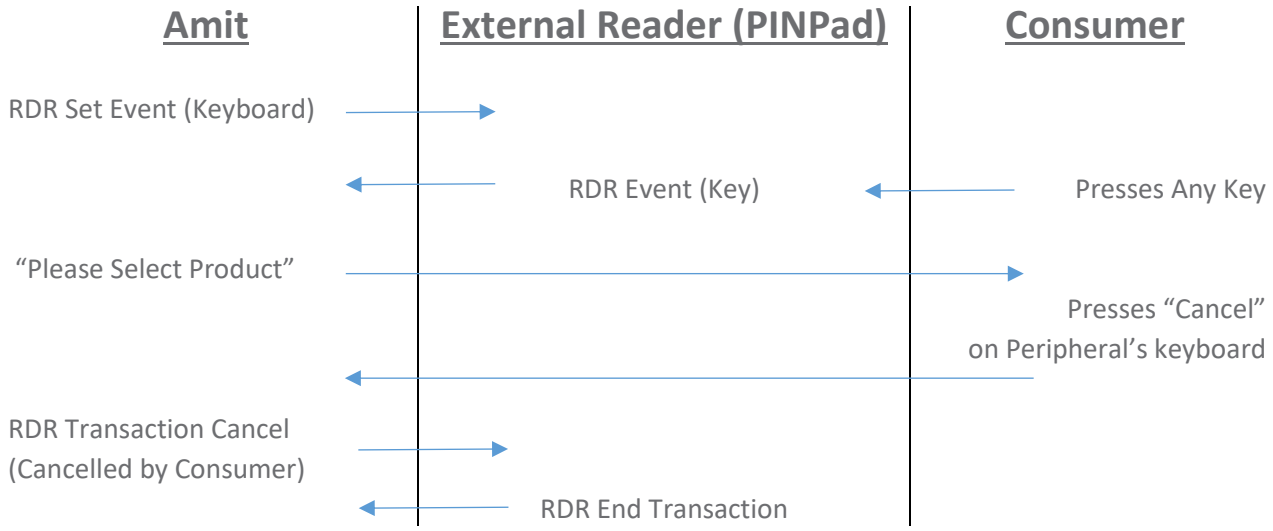
15.2.1 Valid vend



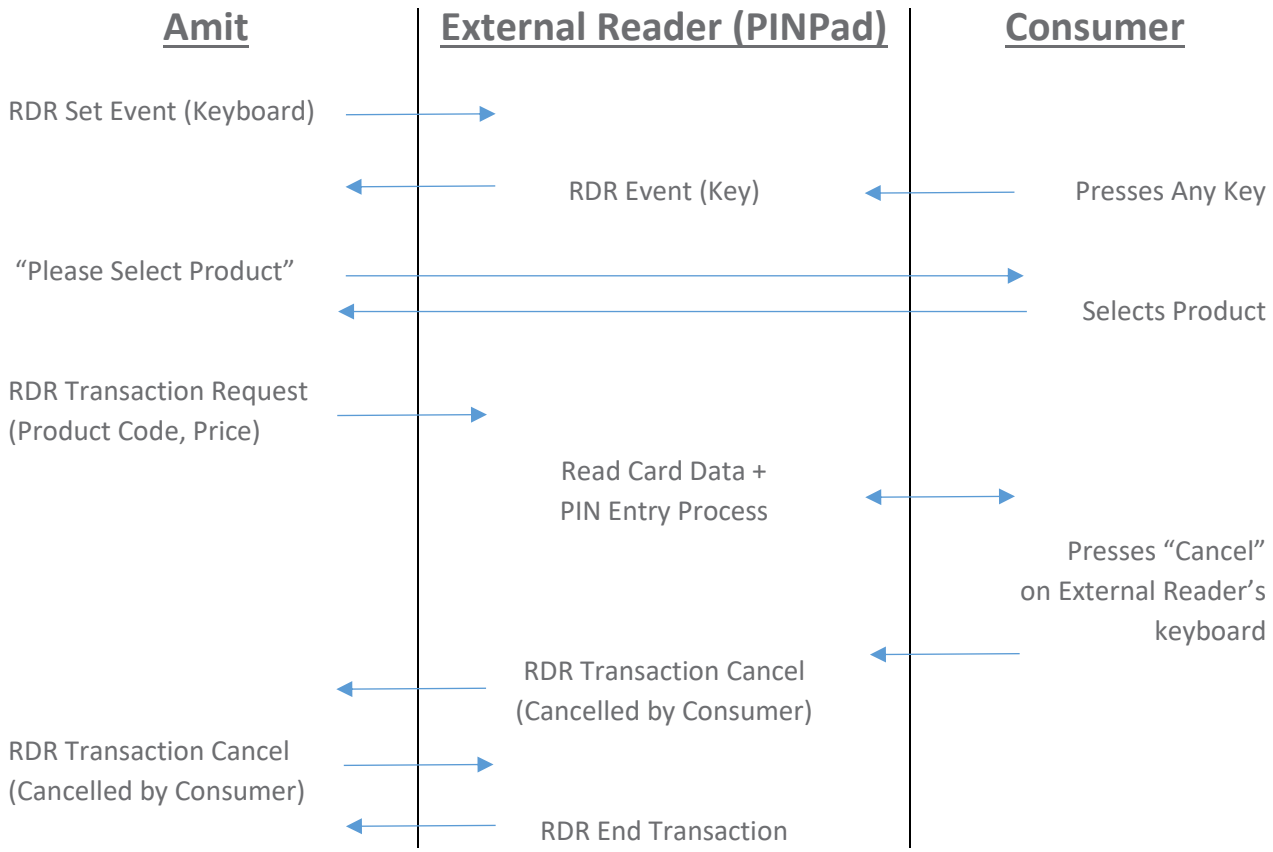
15.2.2 Session cancelled by consumer with External Reader's "Cancel" button



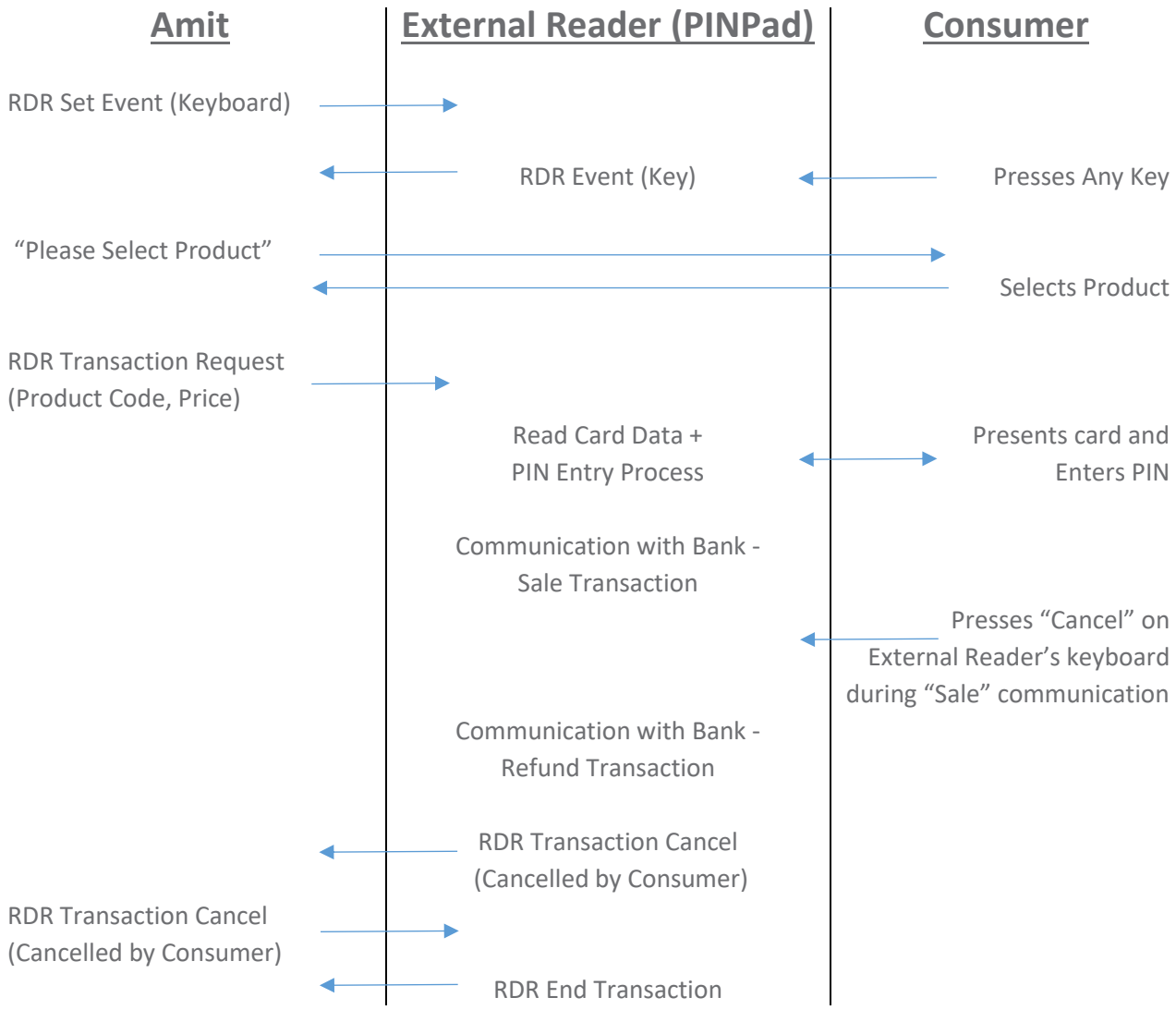
15.2.3 Session cancelled by consumer with Peripheral's "Cancel" button



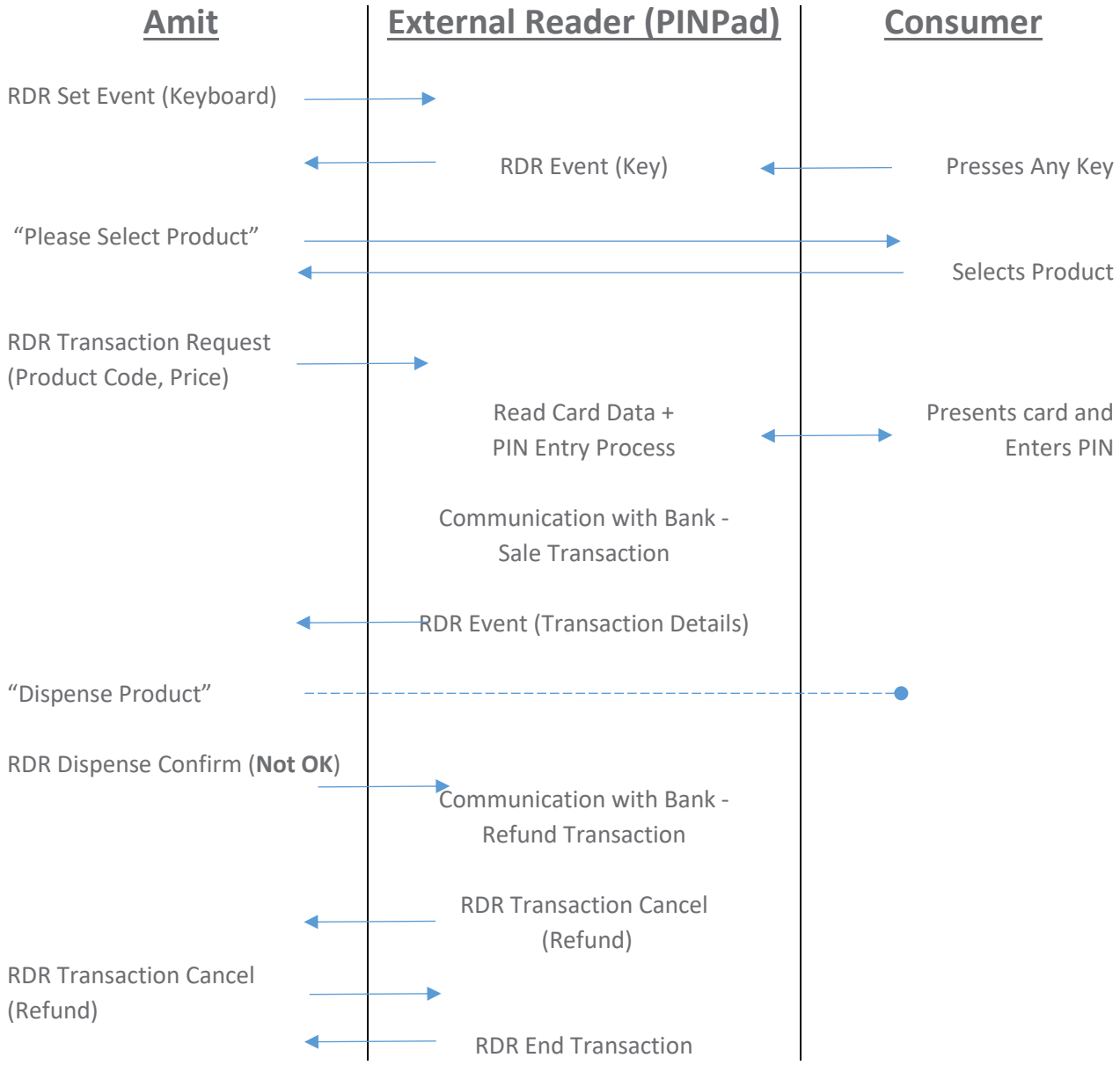
15.2.4 Session cancelled by consumer after "RDR Transaction Request" command



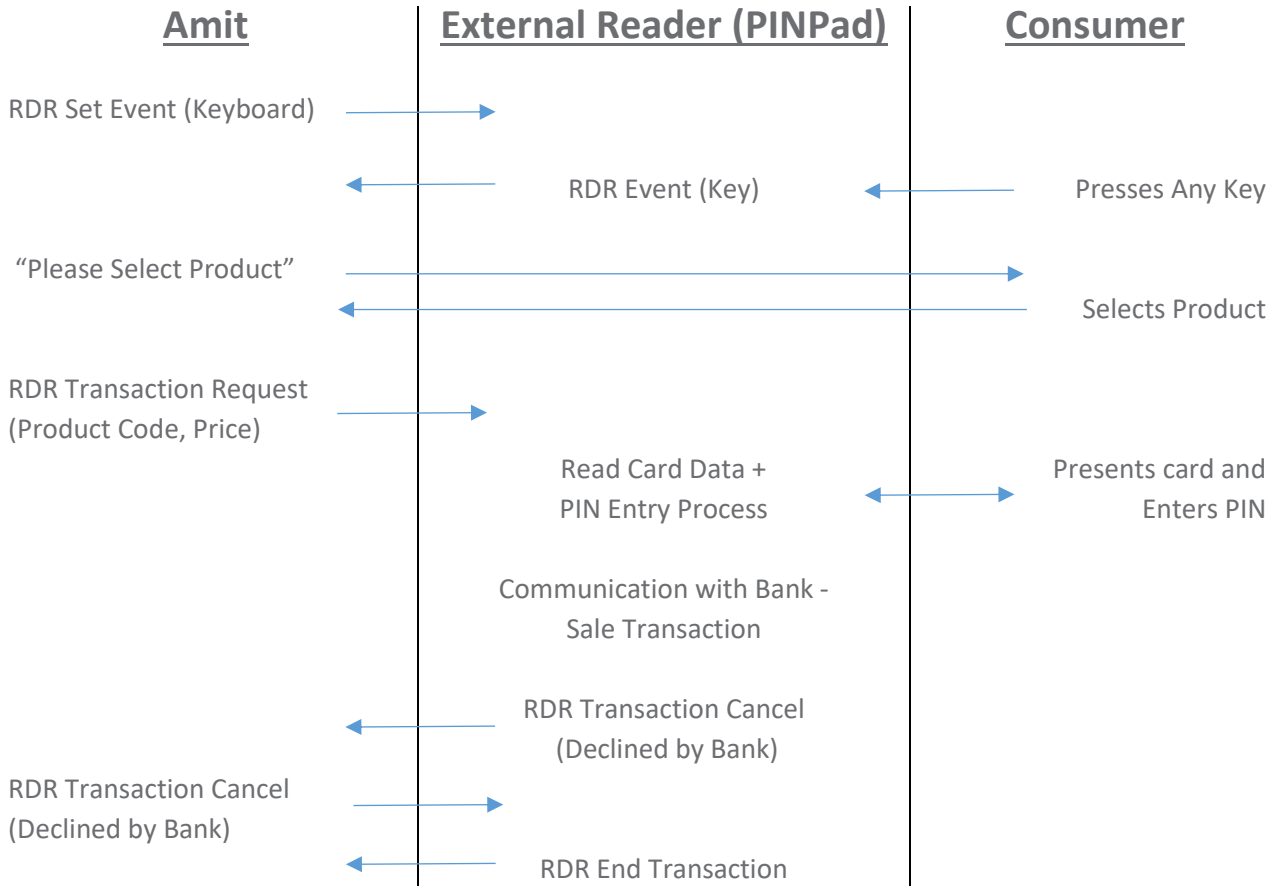
15.2.5 Session cancelled by consumer during "Sale" communication with Bank



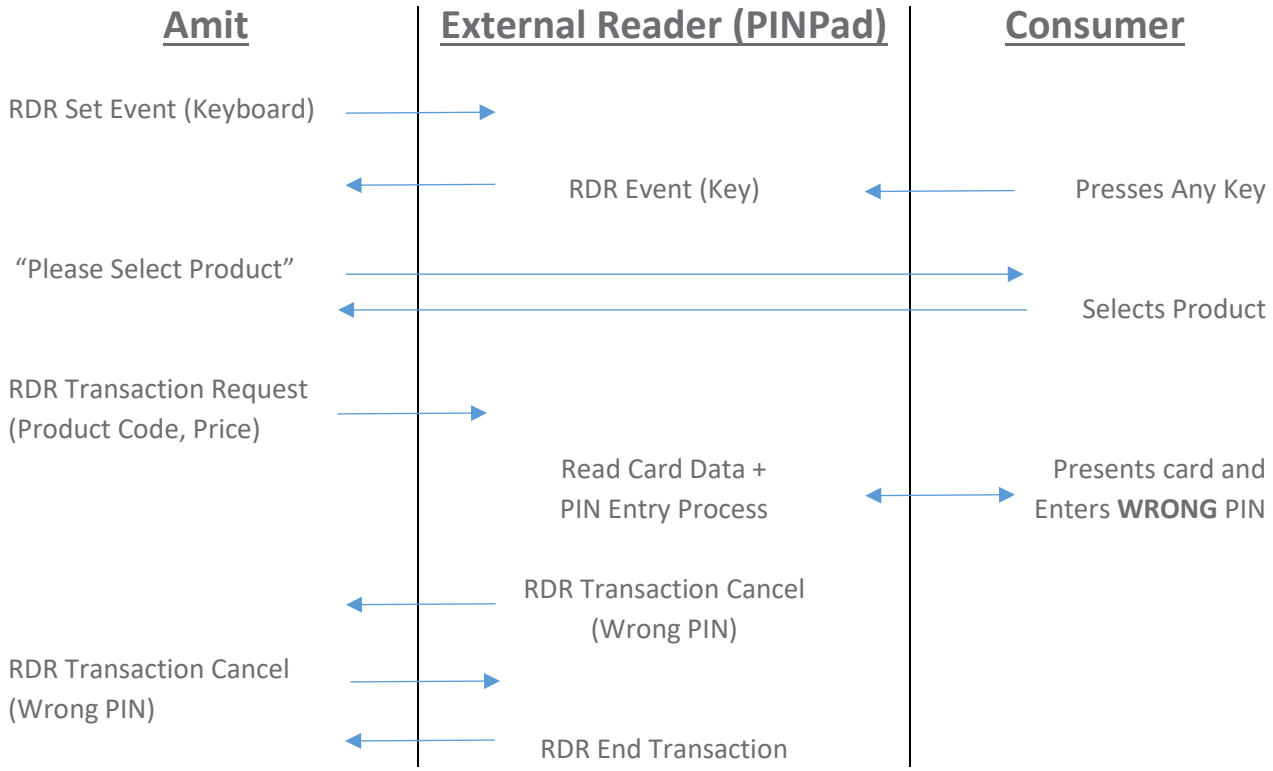
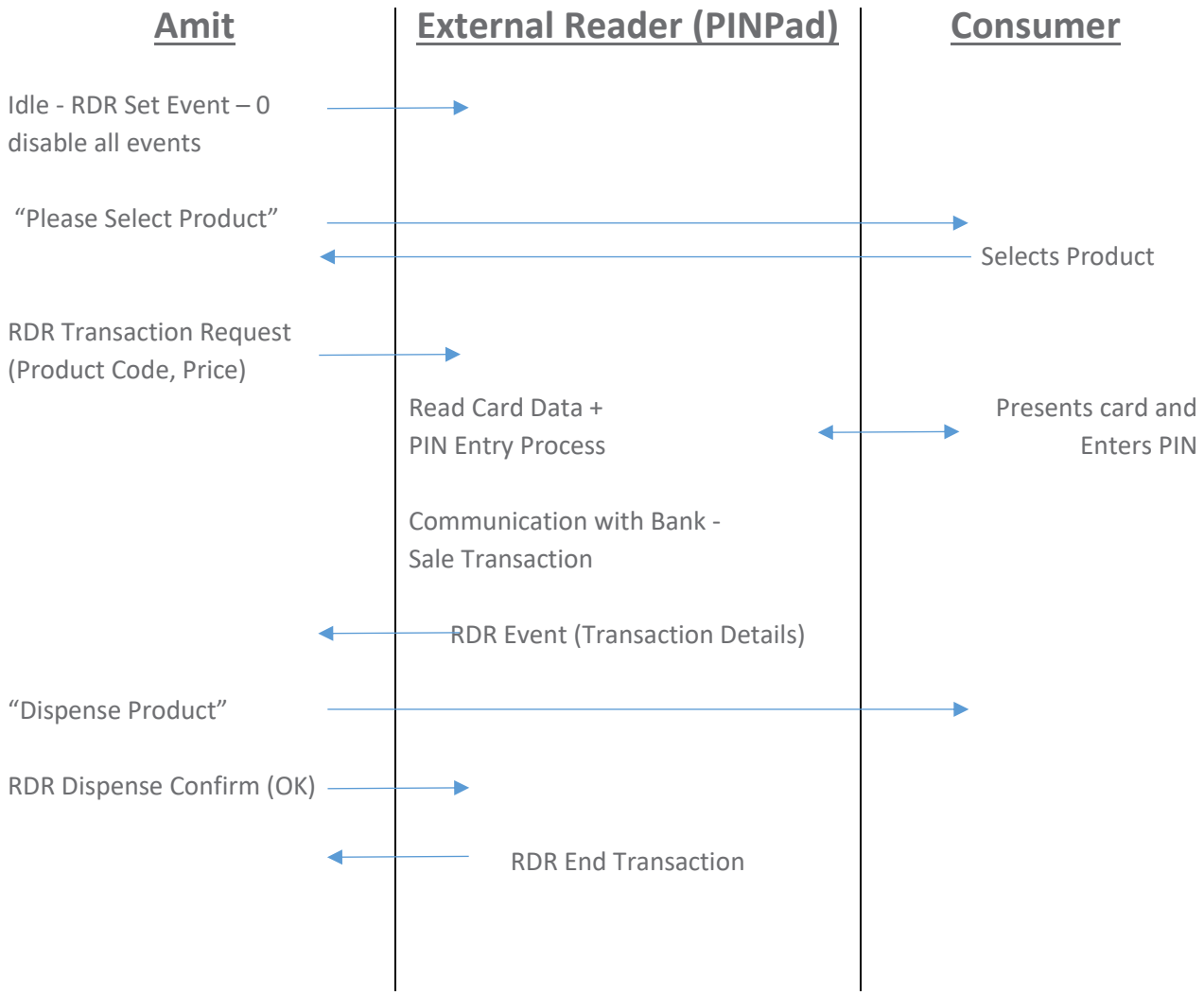
15.2.6 Peripheral failed to dispense product



15.2.7 Vend declined by Bank (Invalid Card, Insufficient fund, etc.)



15.2.8 Session cancelled – Wrong PIN



15.2.9 Always idle (Start with product selection) - Valid vend